# Distributed Systems

## DAT520 - Spring 2026

## Chapter 3 - Reliable Broadcast

Prof. Hein Meling

University
of Stavanger

# Motivation for Reliable Broadcast

# Reliable Broadcast

## Broadcast Abstraction

- Allow processes to *send messages to a group of processes*
- All *correct processes* should **agree on the messages they deliver**
- Key idea
  - Reliability is a **group property**, not just point-to-point delivery
- Why this matters
  - Underlying networks do not provide group-level guarantees
  - Applications must not see diverging views of the system

## Broadcast Abstraction: Where is it Needed?

- Event Dissemination
  - Processes **subscribe to events published** by others
  - No subscriber should miss an update that others observe
- Example
  - Publishing stock quotes to a large number of subscribers
  - All subscribers must see the same sequence of updates

## Broadcast Abstraction: Where is it Needed?

- Replication
  - Servers replicate client requests
  - All replicas must see the same set of requests
- Why broadcast?
  - One client request → many replicas
  - Broadcast abstracts away repeated point-to-point sends

# Reliable Broadcast

## Broadcast Abstraction: A naïve solution

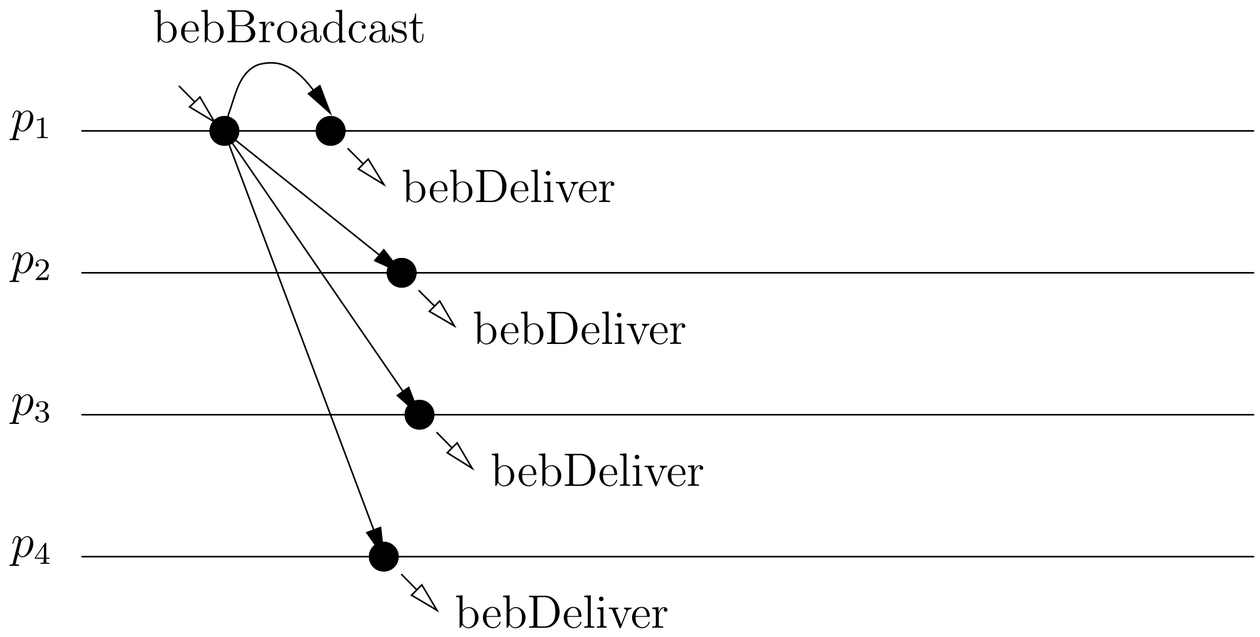
- Use **TCP's reliable point-to-point** channels
- Sender sends the **same message to all processes**
- Bad assumption
  - *If TCP is reliable, broadcast must be reliable too*
- Reality
  - Point-to-point reliability ≠ group reliability

# Best-effort Broadcast

# Ch 3.2 Best-effort Broadcast

## Definition

- Guarantee **all correct** processes **deliver same messages** if **sender correct**
- No delivery guarantee if sender fails



# Ch 3.2 Best-effort Broadcast

**Module 3.1:** Interface and properties of best-effort broadcast

**Module:**

**Name:** BestEffortBroadcast, **instance** *beb*.

**Events:**

**Request:** $\langle$ *beb*, *Broadcast* | $m$ $\rangle$: Broadcasts a message $m$ to all processes.

**Indication:** $\langle$ *beb*, *Deliver* | $p$, $m$ $\rangle$: Delivers a message $m$ broadcast by process $p$.

**Properties:**

**BEB1:** *Validity:* If a correct process broadcasts a message $m$, then every correct process eventually delivers $m$.

**BEB2:** *No duplication:* No message is delivered more than once.

**BEB3:** *No creation:* If a process delivers a message $m$ with sender $s$, then $m$ was previously broadcast by process $s$.

# Ch 3.2 Best-effort Broadcast

- Fail-silent model
- No failure detection
- Sender crash may prevent deliveries

**Algorithm 3.1:** Basic Broadcast

**Implements:**

BestEffortBroadcast, **instance** *beb*.

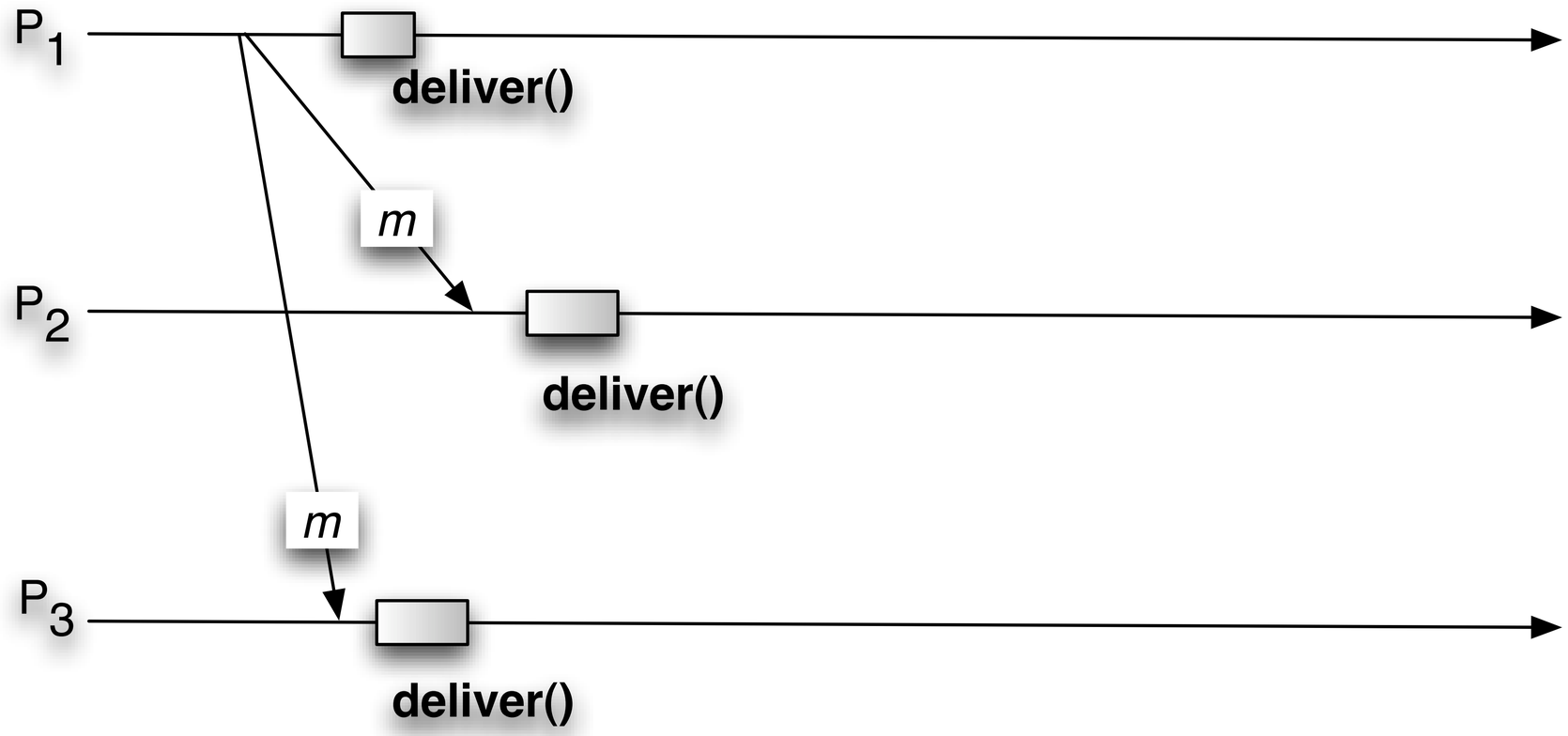
**Uses:**

PerfectPointToPointLinks, **instance** *pl*.

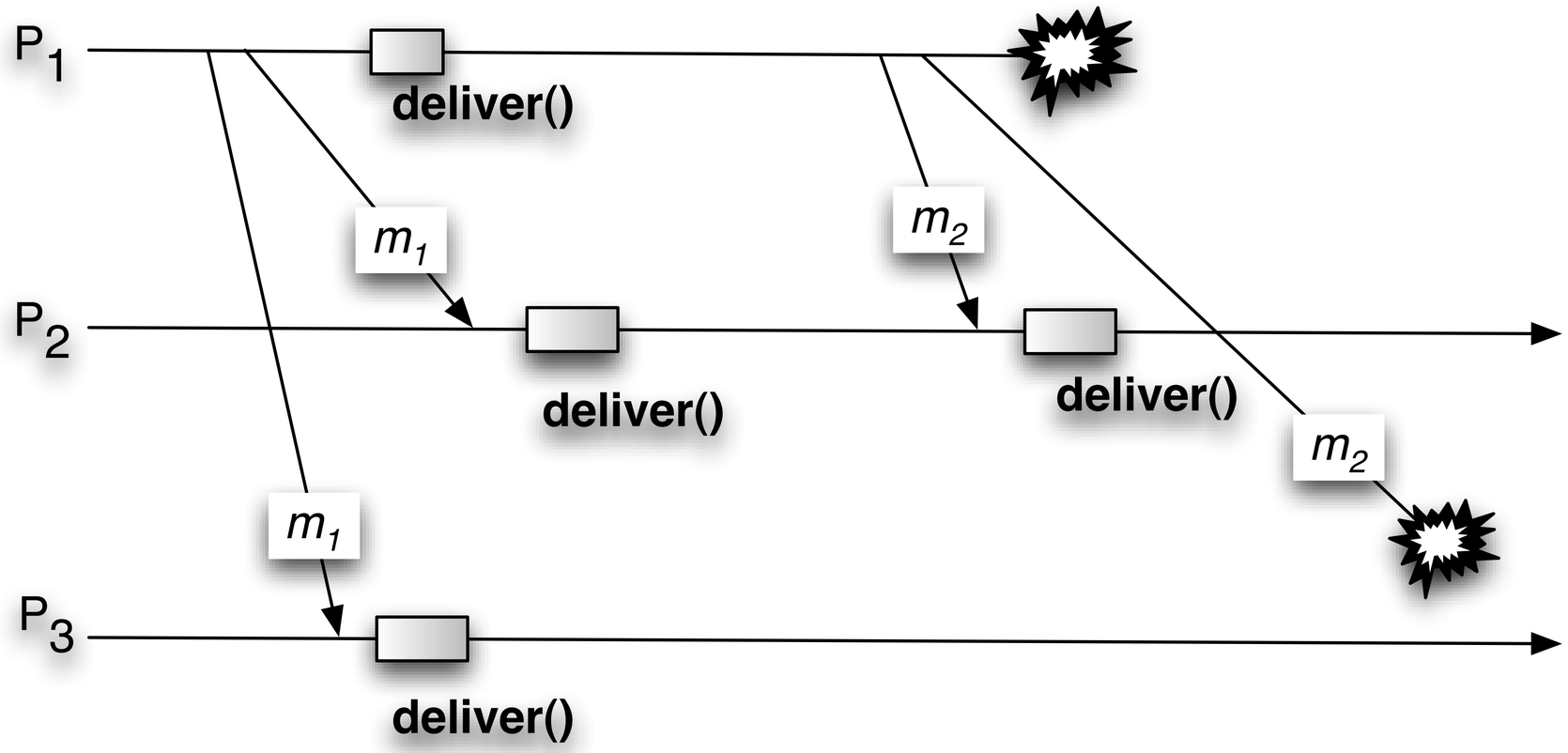
**upon event** $\langle\ beb, Broadcast \mid m\ \rangle$ **do**
  **forall** $q \in \Pi$ **do**
    **trigger** $\langle\ pl, Send \mid q, m\ \rangle$;

**upon event** $\langle\ pl, Deliver \mid p, m\ \rangle$ **do**
  **trigger** $\langle\ beb, Deliver \mid p, m\ \rangle$;

# Ch 3.2 Best-effort Broadcast



# Ch 3.2 Best-effort Broadcast



# Joke time!

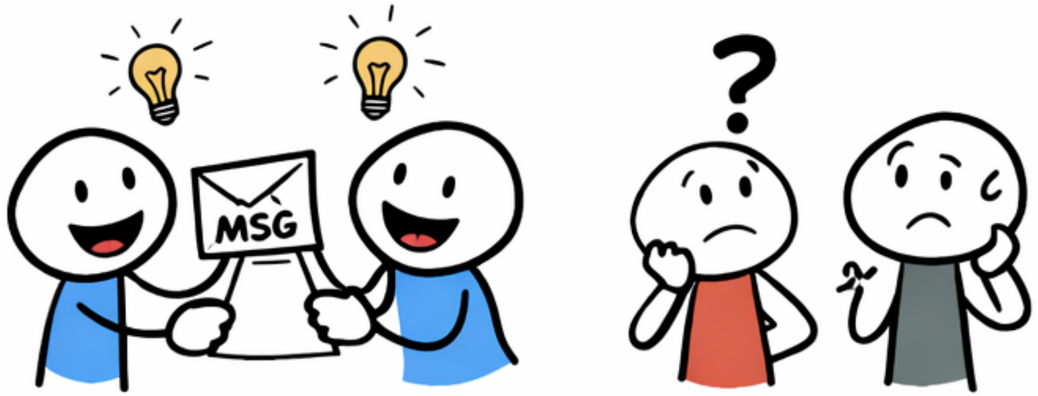
I sent it 

# Broadcast ≠ delivery

# Some heard me. Probably.


What happens next is not my problem.


Into the network void...



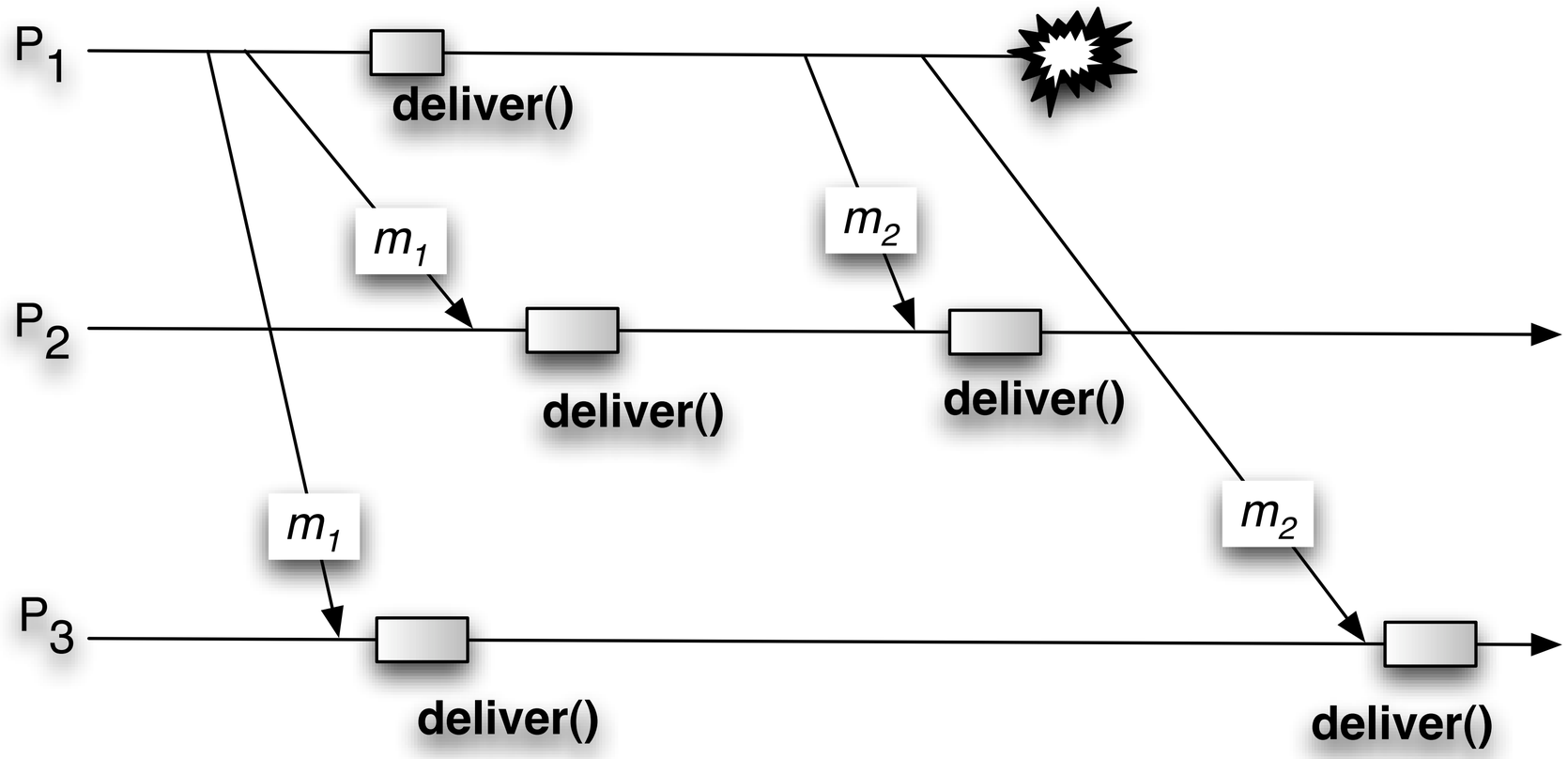

Some got it... Some didn't.


Local success, global mess.

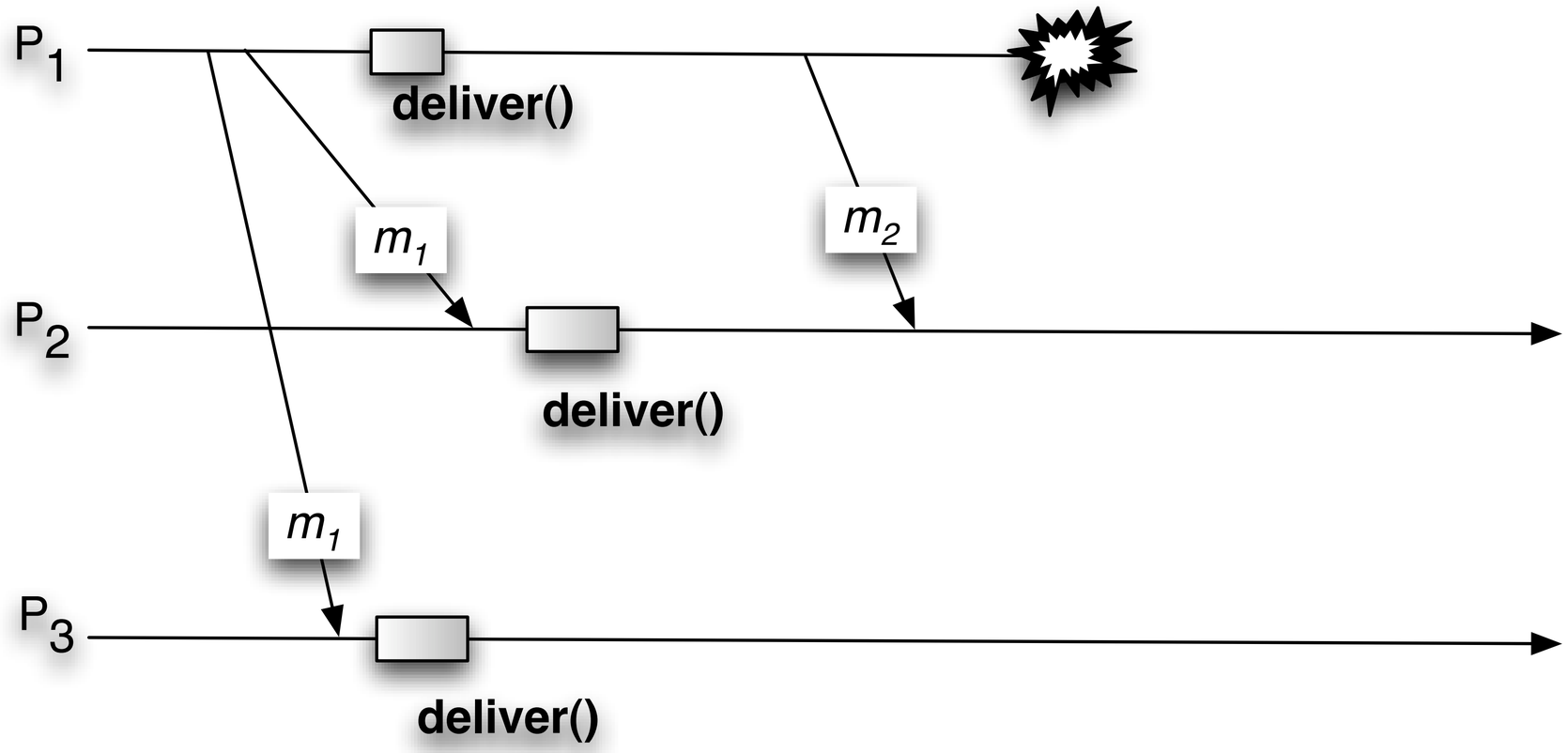
# Regular Reliable Broadcast

# Strengthen Broadcast with Agreement

# 3.3 Regular Reliable Broadcast



# 3.3 Regular Reliable Broadcast



# 3.3 Regular Reliable Broadcast

## Objective

- BEB may disagree if sender fails
- Regular RB: **agreement among correct processes**

# 3.3 Regular Reliable Broadcast

**Module 3.2:** Interface and properties of (regular) reliable broadcast

**Module:**

**Name:** ReliableBroadcast, **instance** *rb*.

**Events:**

**Request:** $\langle$ *rb*, *Broadcast* | $m$ $\rangle$: Broadcasts a message $m$ to all processes.

**Indication:** $\langle$ *rb*, *Deliver* | $p$, $m$ $\rangle$: Delivers a message $m$ broadcast by process $p$.

**Properties:**

Liveness **RB1:** *Validity:* If a correct process $p$ broadcasts a message $m$, then $p$ eventually delivers $m$.

Safety **RB2:** *No duplication:* No message is delivered more than once.

Safety **RB3:** *No creation:* If a process delivers a message $m$ with sender $s$, then $m$ was previously broadcast by process $s$.

Liveness **RB4:** *Agreement:* If a message $m$ is delivered by some correct process, then $m$ is eventually delivered by every correct process.

# Two Regular RB Algorithms

# 3.3 Regular Reliable Broadcast

- Fail-stop model
- Uses perfect failure detector
- Retransmit on crash

**Algorithm 3.2:** Lazy Reliable Broadcast

**Implements:**
ReliableBroadcast, **instance** *rb*.

**Uses:**
BestEffortBroadcast, **instance** *beb*;
PerfectFailureDetector, **instance** $\mathcal{P}$.

**upon event** $\langle$ *rb*, *Init* $\rangle$ **do**
*correct* := $\Pi$;
*from*[$p$] := $[\emptyset]^N$;

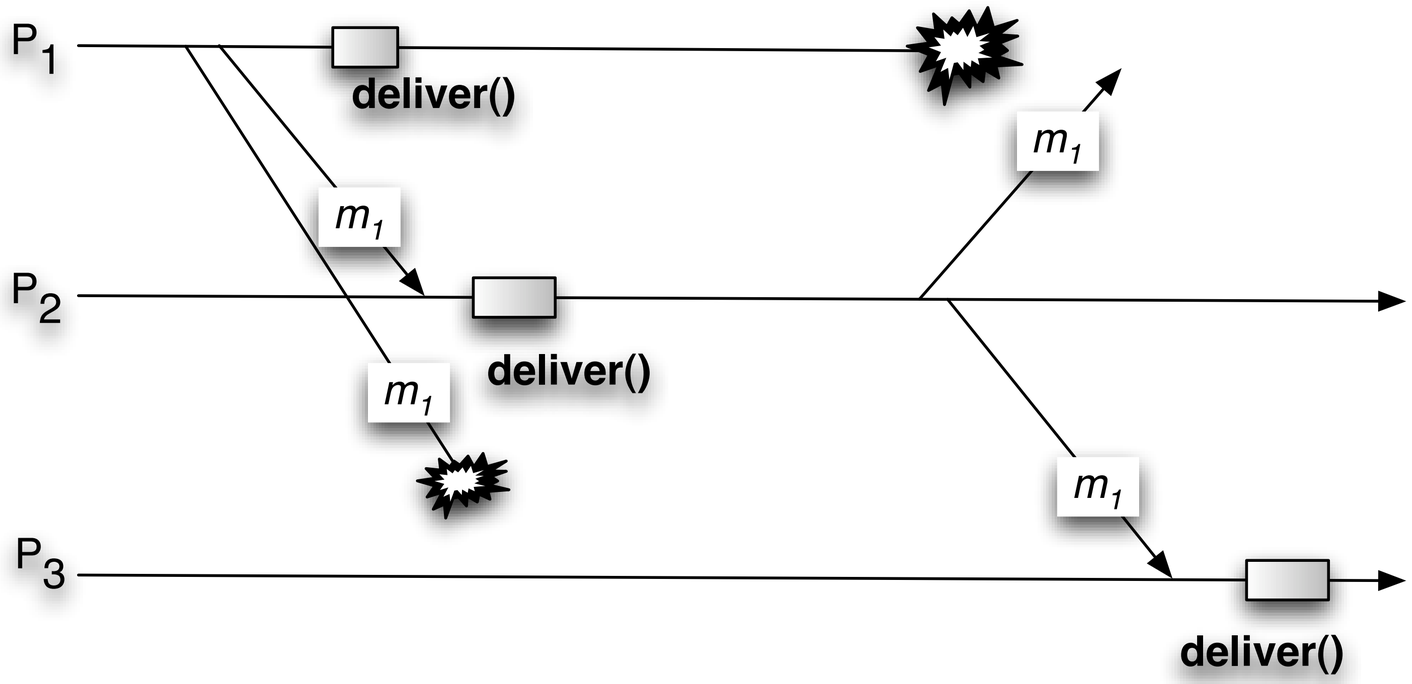
**upon event** $\langle$ *rb*, *Broadcast* | $m$ $\rangle$ **do**
**trigger** $\langle$ *beb*, *Broadcast* | [DATA, *self*, $m$] $\rangle$;

**upon event** $\langle$ *beb*, *Deliver* | $p$, [DATA, $s, m$] $\rangle$ **do**
**if** $m \notin$ *from*[$s$] **then**
**trigger** $\langle$ *rb*, *Deliver* | $s, m$ $\rangle$;
*from*[$s$] := *from*[$s$] $\cup \{m\}$;
**if** $s \notin$ *correct* **then**
**trigger** $\langle$ *beb*, *Broadcast* | [DATA, $s, m$] $\rangle$;

**upon event** $\langle$ $\mathcal{P}$, *Crash* | $p$ $\rangle$ **do**
*correct* := *correct* $\setminus \{p\}$;
**forall** $m \in$ *from*[$p$] **do**
**trigger** $\langle$ *beb*, *Broadcast* | [DATA, $p, m$] $\rangle$;

# 3.3 Regular Reliable Broadcast

Whiteboard




**Algorithm 3.2:** Lazy Reliable Broadcast

**Implements:**
ReliableBroadcast, **instance** *rb*.

**Uses:**
BestEffortBroadcast, **instance** *beb*;
PerfectFailureDetector, **instance** $\mathcal{P}$.

**upon event** $\langle$ *rb*, *Init* $\rangle$ **do**
$correct := \Pi$;
$from[p] := [\emptyset]^N$;

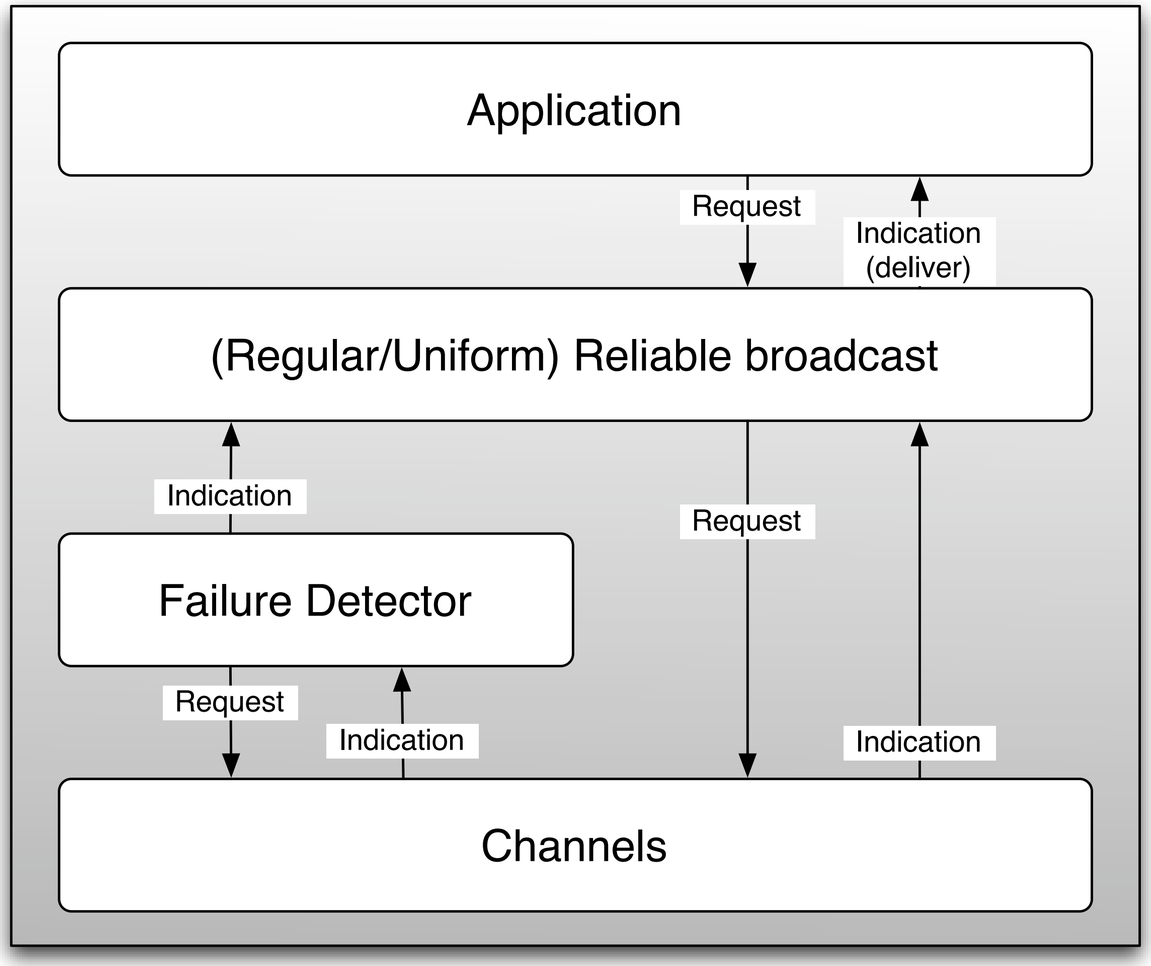
**upon event** $\langle$ *rb*, *Broadcast* | $m$ $\rangle$ **do**
**trigger** $\langle$ *beb*, *Broadcast* | [DATA, *self*, $m$] $\rangle$;

**upon event** $\langle$ *beb*, *Deliver* | $p$, [DATA, $s, m$] $\rangle$ **do**
**if** $m \notin from[s]$ **then**
**trigger** $\langle$ *rb*, *Deliver* | $s, m$ $\rangle$;
$from[s] := from[s] \cup \{m\}$;
**if** $s \notin correct$ **then**
**trigger** $\langle$ *beb*, *Broadcast* | [DATA, $s, m$] $\rangle$;

**upon event** $\langle$ $\mathcal{P}$, *Crash* | $p$ $\rangle$ **do**
$correct := correct \setminus \{p\}$;
**forall** $m \in from[p]$ **do**
**trigger** $\langle$ *beb*, *Broadcast* | [DATA, $p, m$] $\rangle$;

# 3.3 Regular Reliable Broadcast

## Whiteboard




**Algorithm 3.2:** Lazy Reliable Broadcast

**Implements:**
ReliableBroadcast, **instance** *rb*.

**Uses:**
BestEffortBroadcast, **instance** *beb*;
PerfectFailureDetector, **instance** $\mathcal{P}$.

**upon event** $\langle$ *rb*, *Init* $\rangle$ **do**
$correct := \Pi$;
$from[p] := [\emptyset]^N$;

**upon event** $\langle$ *rb*, *Broadcast* | $m$ $\rangle$ **do**
**trigger** $\langle$ *beb*, *Broadcast* | [DATA, *self*, $m$] $\rangle$;

**upon event** $\langle$ *beb*, *Deliver* | $p$, [DATA, $s, m$] $\rangle$ **do**
**if** $m \notin from[s]$ **then**
**trigger** $\langle$ *rb*, *Deliver* | $s, m$ $\rangle$;
$from[s] := from[s] \cup \{m\}$;
**if** $s \notin correct$ **then**
**trigger** $\langle$ *beb*, *Broadcast* | [DATA, $s, m$] $\rangle$;

**upon event** $\langle$ $\mathcal{P}$, *Crash* | $p$ $\rangle$ **do**
$correct := correct \setminus \{p\}$;
**forall** $m \in from[p]$ **do**
**trigger** $\langle$ *beb*, *Broadcast* | [DATA, $p, m$] $\rangle$;

# 3.3 Regular Reliable Broadcast

- Fail-silent model
- No failure detection
- Proactive rebroadcast

**Algorithm 3.3:** Eager Reliable Broadcast

**Implements:**
ReliableBroadcast, **instance** *rb*.

**Uses:**
BestEffortBroadcast, **instance** *beb*.

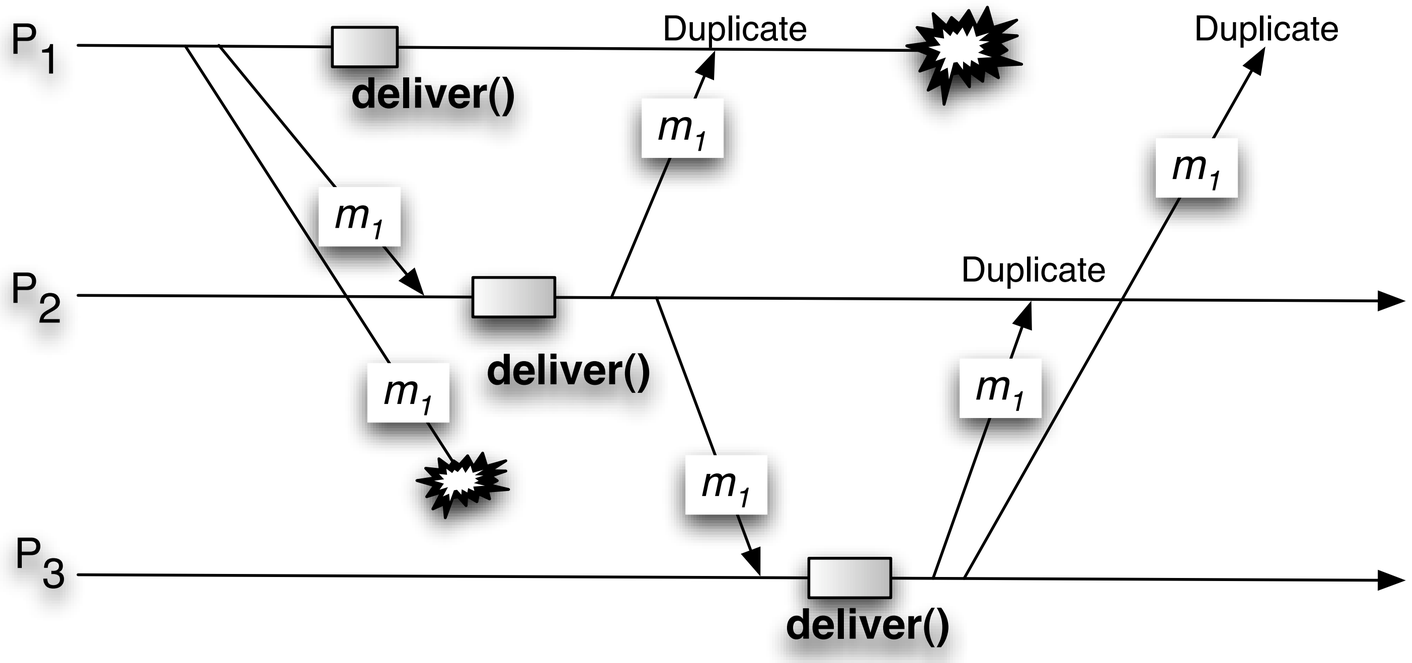
**upon event** $\langle$ *rb*, *Init* $\rangle$ **do**
*delivered* := $\emptyset$;

**upon event** $\langle$ *rb*, *Broadcast* | $m$ $\rangle$ **do**
**trigger** $\langle$ *beb*, *Broadcast* | [DATA, *self*, $m$] $\rangle$;

**upon event** $\langle$ *beb*, *Deliver* | $p$, [DATA, $s, m$] $\rangle$ **do**
**if** $m \notin$ *delivered* **then**
*delivered* := *delivered* $\cup \{m\}$;
**trigger** $\langle$ *rb*, *Deliver* | $s, m$ $\rangle$;
**trigger** $\langle$ *beb*, *Broadcast* | [DATA, $s, m$] $\rangle$;

# 3.3 Regular Reliable Broadcast

Whiteboard




**Algorithm 3.3:** Eager Reliable Broadcast

**Implements:**
ReliableBroadcast, **instance** *rb*.

**Uses:**
BestEffortBroadcast, **instance** *beb*.

**upon event** $\langle$ *rb*, *Init* $\rangle$ **do**
*delivered* := $\emptyset$;

**upon event** $\langle$ *rb*, *Broadcast* | $m$ $\rangle$ **do**
**trigger** $\langle$ *beb*, *Broadcast* | [DATA, *self*, $m$] $\rangle$;

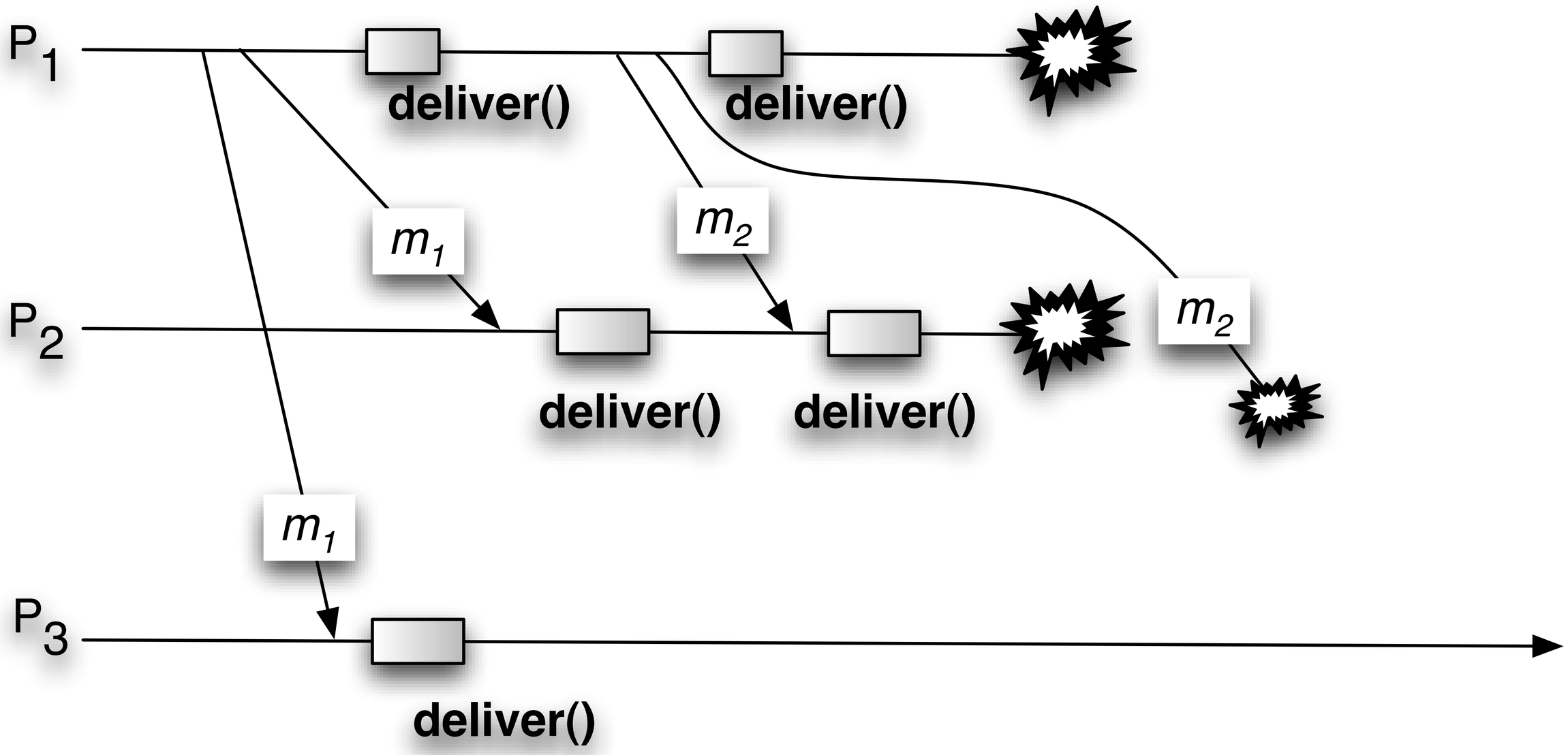
**upon event** $\langle$ *beb*, *Deliver* | $p$, [DATA, $s$, $m$] $\rangle$ **do**
**if** $m \notin$ *delivered* **then**
*delivered* := *delivered* $\cup \{m\}$;
**trigger** $\langle$ *rb*, *Deliver* | $s$, $m$ $\rangle$;
**trigger** $\langle$ *beb*, *Broadcast* | [DATA, $s$, $m$] $\rangle$;

# Uniform Reliable Broadcast

# The Problem with Regular Reliable Broadcast

# No Guarantees for Faulty Processes

# Ch 3.4 Uniform Reliable Broadcast

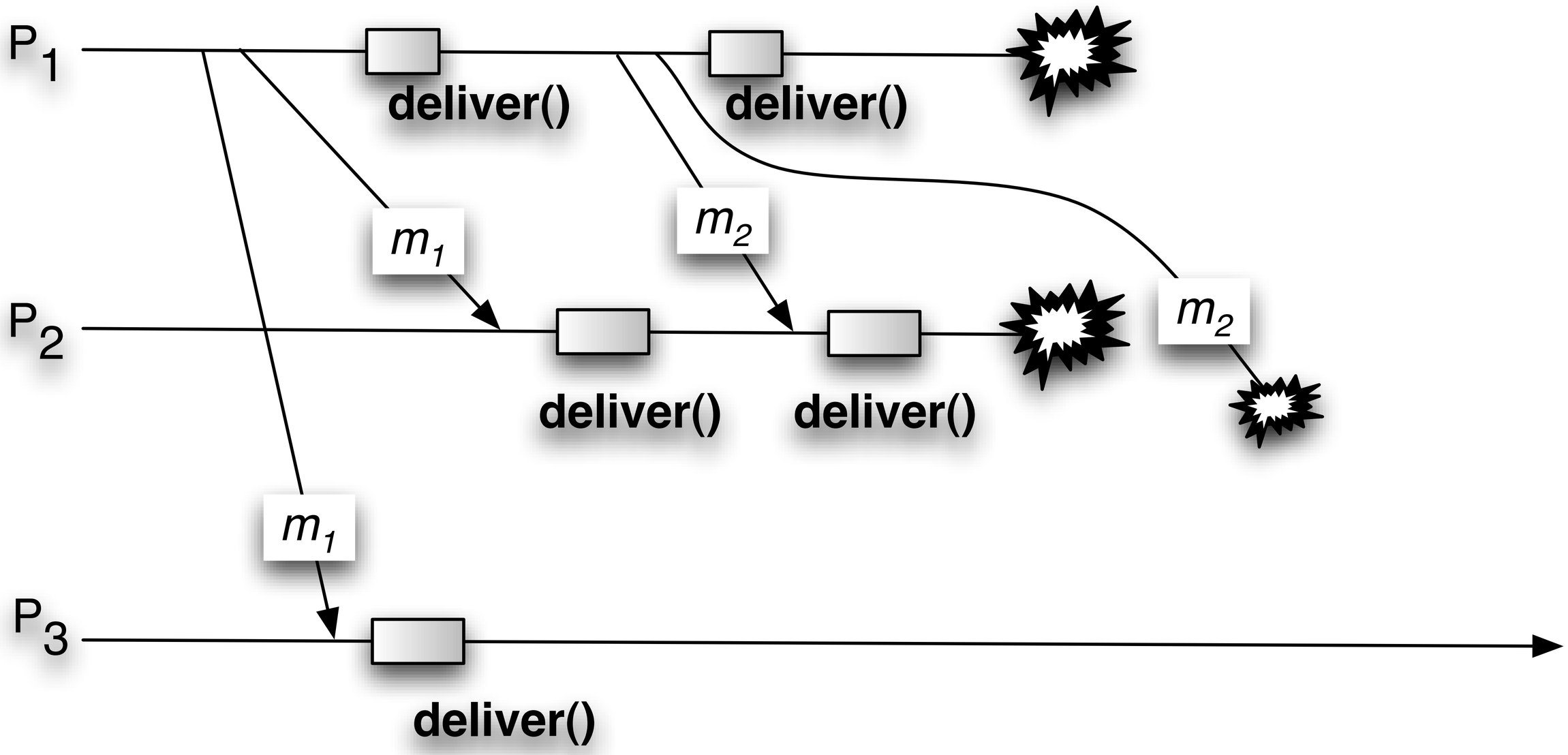


# Ch 3.4 Uniform Reliable Broadcast

## Uniformity Gap

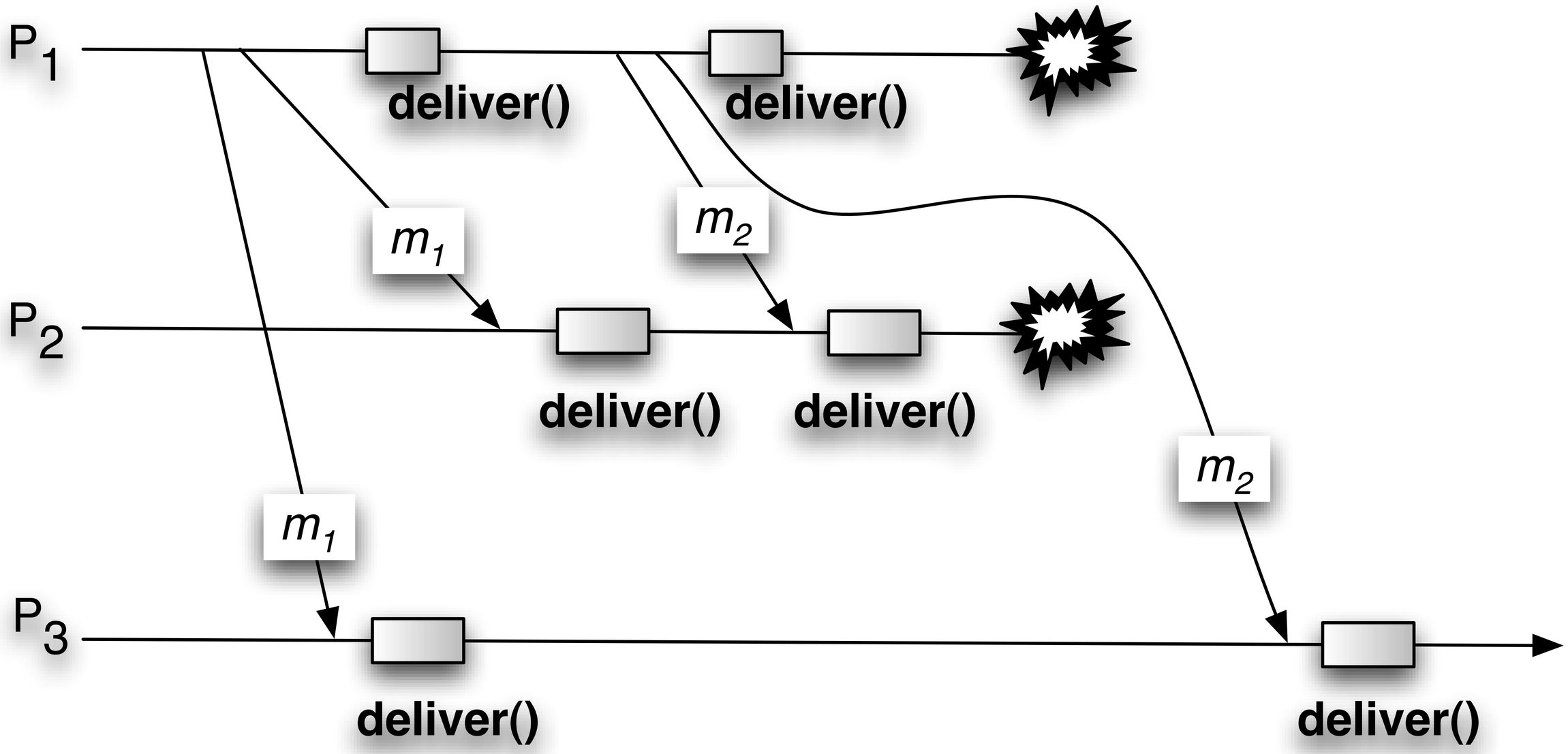
- No guarantees for faulty processes
- Regular RB ignores deliveries by crashed processes
- This gap can lead to **inconsistent external actions**
  - Trigger rocket launch!

# Ch 3.4 Uniform Reliable Broadcast

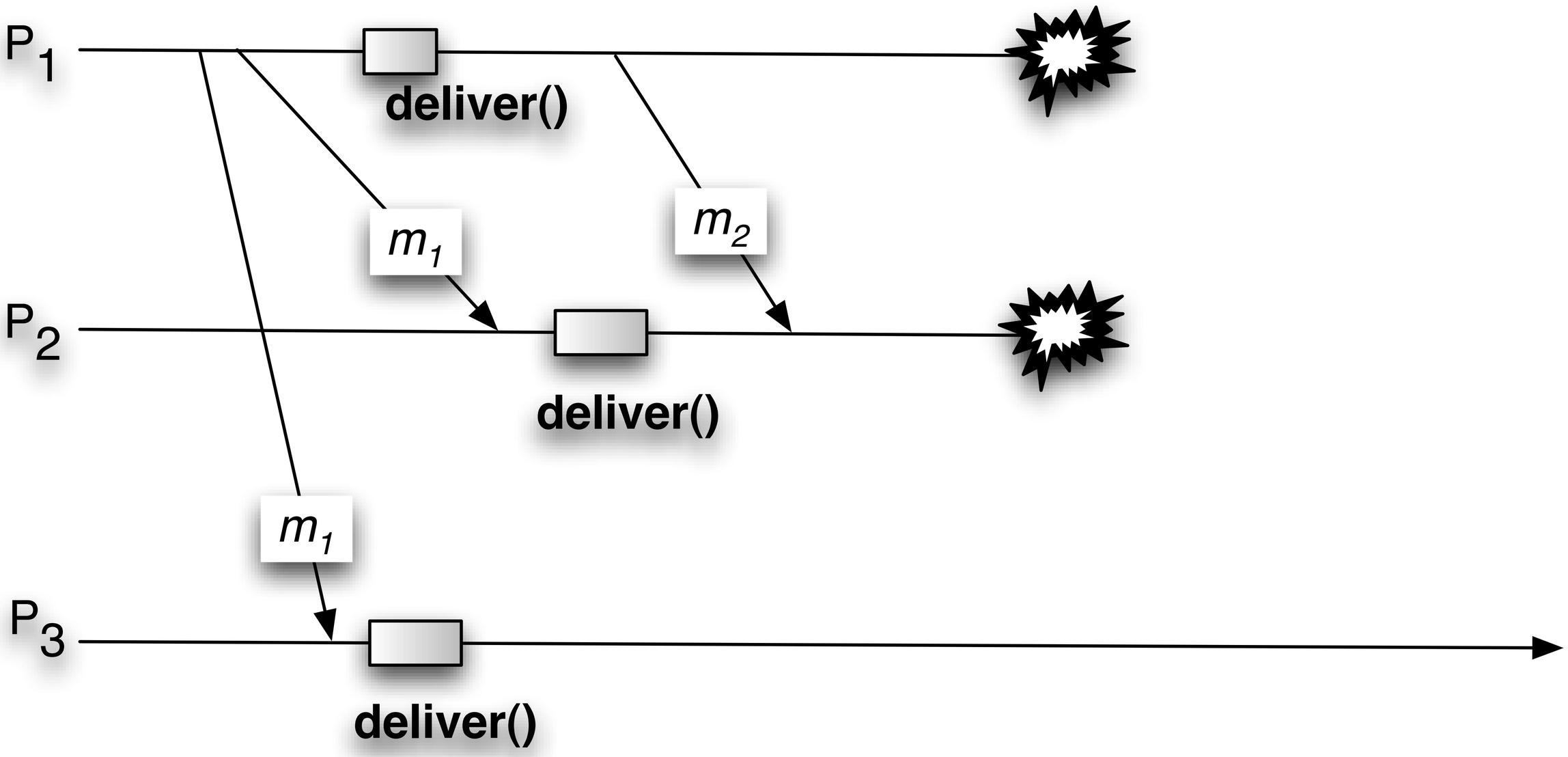


# Strengthen Reliable Broadcast with Uniform Delivery

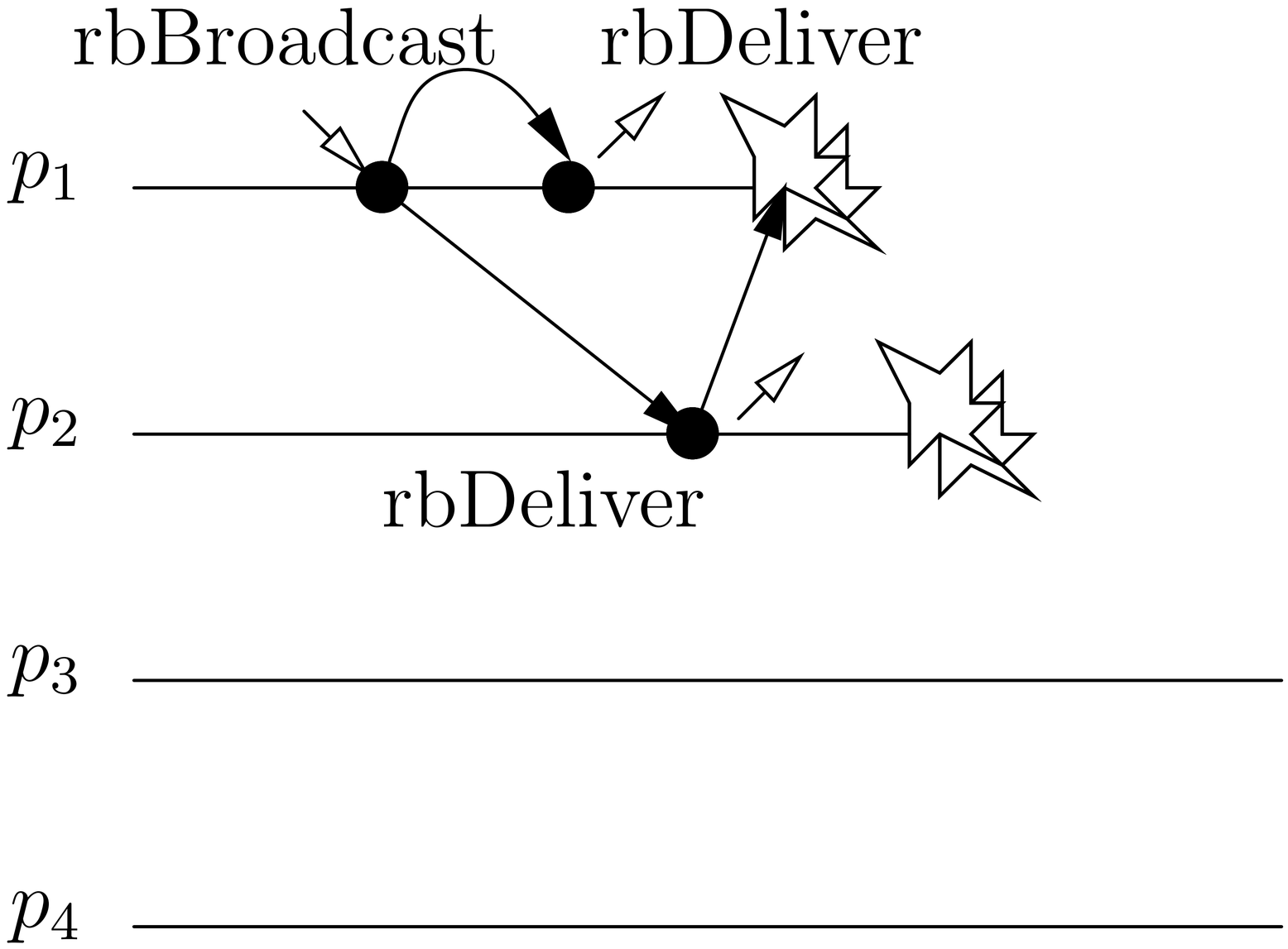
# Ch 3.4 Uniform Reliable Broadcast



# Ch 3.4 Uniform Reliable Broadcast



# Ch 3.4 Uniform Reliable Broadcast



# Ch 3.4 Uniform Reliable Broadcast

**Module 3.3:** Interface and properties of uniform reliable broadcast

**Module:**

**Name:** UniformReliableBroadcast, **instance** *urb*.

**Events:**

**Request:** $\langle$ *urb*, *Broadcast* | $m$ $\rangle$: Broadcasts a message $m$ to all processes.

**Indication:** $\langle$ *urb*, *Deliver* | $p$, $m$ $\rangle$: Delivers a message $m$ broadcast by process $p$.

**Properties:**

**URB1–URB3:** Same as properties RB1–RB3 in (regular) reliable broadcast (Module 3.2).

**URB4:** *Uniform agreement:* If a message $m$ is delivered by some process (whether correct or faulty), then $m$ is eventually delivered by every correct process.

# Ch 3.4 Uniform Reliable Broadcast

- Fail-stop model
- Uses perfect failure detector
- Deliver only after **all correct** have seen
- Relay once process see the message

**Algorithm 3.4:** All-Ack Uniform Reliable Broadcast

**Implements:**
UniformReliableBroadcast, **instance** *urb*.

**Uses:**
BestEffortBroadcast, **instance** *beb*.
PerfectFailureDetector, **instance** $\mathcal{P}$.

**upon event** $\langle$ *urb*, *Init* $\rangle$ **do**
*delivered* := $\emptyset$;
*pending* := $\emptyset$;
*correct* := $\Pi$;
**forall** $m$ **do** *ack*$[m]$ := $\emptyset$;

**upon event** $\langle$ *urb*, *Broadcast* | $m$ $\rangle$ **do**
*pending* := *pending* $\cup$ $\{(self, m)\}$;
**trigger** $\langle$ *beb*, *Broadcast* | [DATA, *self*, $m$] $\rangle$;

**upon event** $\langle$ *beb*, *Deliver* | $p$, [DATA, $s, m$] $\rangle$ **do**
*ack*$[m]$ := *ack*$[m]$ $\cup$ $\{p\}$;
**if** $(s, m) \notin$ *pending* **then**
*pending* := *pending* $\cup$ $\{(s, m)\}$;
**trigger** $\langle$ *beb*, *Broadcast* | [DATA, $s, m$] $\rangle$;

**upon event** $\langle$ $\mathcal{P}$, *Crash* | $p$ $\rangle$ **do**
*correct* := *correct* $\setminus$ $\{p\}$;

**function** *candeliver*$(m)$ **returns** Boolean **is**
**return** $(correct \subseteq ack[m])$;

**upon exists** $(s, m) \in$ *pending* such that *candeliver*$(m) \wedge m \notin$ *delivered* **do**
*delivered* := *delivered* $\cup$ $\{m\}$;
**trigger** $\langle$ *urb*, *Deliver* | $s, m$ $\rangle$;

# Ch 3.4 Uniform Reliable Broadcast

Whiteboard

**upon event** $\langle$ *urb*, *Init* $\rangle$ **do**
    $delivered := \emptyset$;
    $pending := \emptyset$;
    $correct := \Pi$;
    **forall** $m$ **do** $ack[m] := \emptyset$;

**upon event** $\langle$ *urb*, *Broadcast* | $m$ $\rangle$ **do**
    $pending := pending \cup \{(self, m)\}$;
    **trigger** $\langle$ *beb*, *Broadcast* | [DATA, *self*, $m$] $\rangle$;

**upon event** $\langle$ *beb*, *Deliver* | $p$, [DATA, $s, m$] $\rangle$ **do**
    $ack[m] := ack[m] \cup \{p\}$;
    **if** $(s, m) \notin pending$ **then**
        $pending := pending \cup \{(s, m)\}$;
        **trigger** $\langle$ *beb*, *Broadcast* | [DATA, $s, m$] $\rangle$;

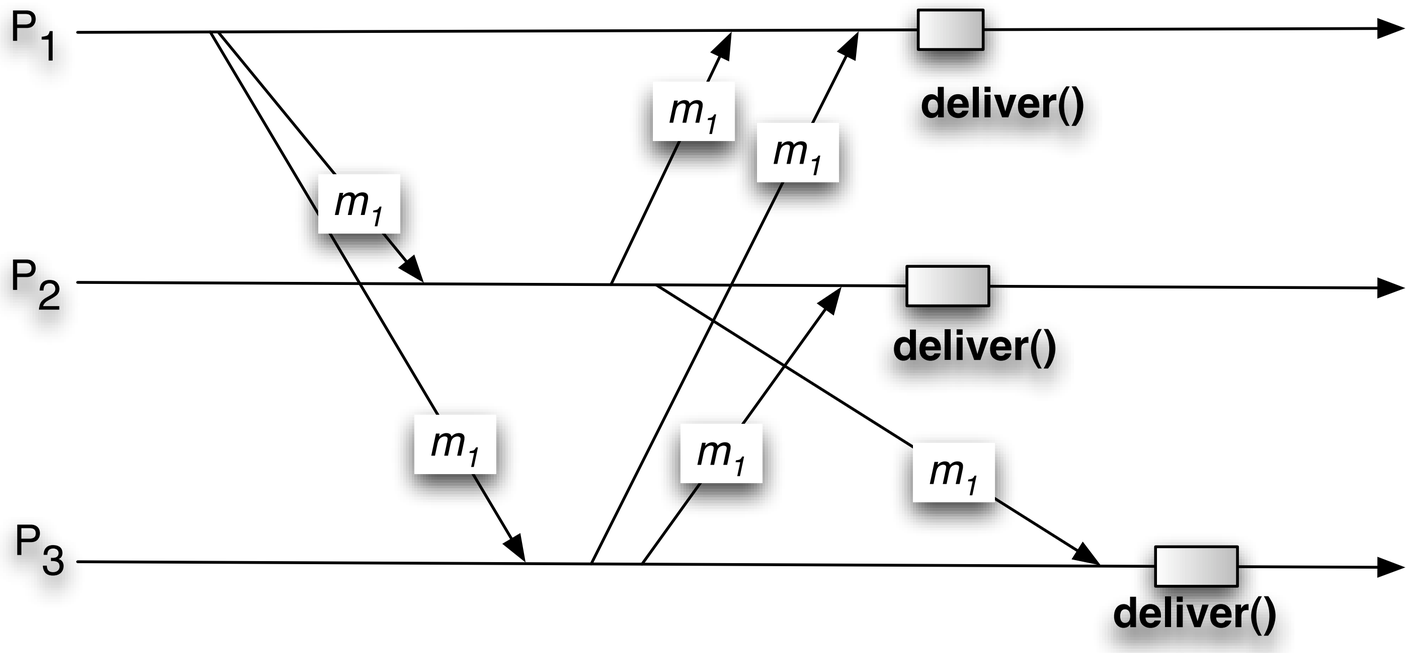
**upon event** $\langle$ $\mathcal{P}$, *Crash* | $p$ $\rangle$ **do**
    $correct := correct \setminus \{p\}$;

**function** *candeliver*($m$) **returns** Boolean **is**
    **return** $(correct \subseteq ack[m])$;

**upon exists** $(s, m) \in pending$ such that *candeliver*($m$) $\wedge$ $m \notin delivered$ **do**
    $delivered := delivered \cup \{m\}$;
    **trigger** $\langle$ *urb*, *Deliver* | $s$, $m$ $\rangle$;

# Ch 3.4 Uniform Reliable Broadcast

## Whiteboard




**Algorithm 3.4:** All-Ack Uniform Reliable Broadcast

**Implements:**
UniformReliableBroadcast, **instance** *urb*.

**Uses:**
BestEffortBroadcast, **instance** *beb*.
PerfectFailureDetector, **instance** $\mathcal{P}$.

**upon event** $\langle$ *urb*, *Init* $\rangle$ **do**
*delivered* := $\emptyset$;
*pending* := $\emptyset$;
*correct* := $\Pi$;
**forall** $m$ **do** *ack*$[m]$ := $\emptyset$;

**upon event** $\langle$ *urb*, *Broadcast* | $m$ $\rangle$ **do**
*pending* := *pending* $\cup$ $\{(self, m)\}$;
**trigger** $\langle$ *beb*, *Broadcast* | [DATA, *self*, $m$] $\rangle$;

**upon event** $\langle$ *beb*, *Deliver* | $p$, [DATA, $s, m$] $\rangle$ **do**
*ack*$[m]$ := *ack*$[m]$ $\cup$ $\{p\}$;
**if** $(s, m) \notin$ *pending* **then**
*pending* := *pending* $\cup$ $\{(s, m)\}$;
**trigger** $\langle$ *beb*, *Broadcast* | [DATA, $s, m$] $\rangle$;

**upon event** $\langle$ $\mathcal{P}$, *Crash* | $p$ $\rangle$ **do**
*correct* := *correct* $\setminus$ $\{p\}$;

**function** *candeliver*$(m)$ **returns** Boolean **is**
**return** $(correct \subseteq ack[m])$;

**upon exists** $(s, m) \in$ *pending* such that *candeliver*$(m) \wedge m \notin$ *delivered* **do**
*delivered* := *delivered* $\cup$ $\{m\}$;
**trigger** $\langle$ *urb*, *Deliver* | $s, m$ $\rangle$;

# Ch 3.4 Uniform Reliable Broadcast

- Fail-silent model
- No failure detector
- Deliver once a **majority** have seen
- Relay once process see the message

**Algorithm 3.5:** Majority-Ack Uniform Reliable Broadcast

**Implements:**
UniformReliableBroadcast, **instance** *urb*.

**Uses:**
BestEffortBroadcast, **instance** *beb*.

**upon event** $\langle$ *urb*, *Init* $\rangle$ **do**
*delivered* := $\emptyset$;
*pending* := $\emptyset$;
*correct* := $\Pi$;
**forall** $m$ **do** *ack*$[m]$ := $\emptyset$;

**upon event** $\langle$ *urb*, *Broadcast* | $m$ $\rangle$ **do**
*pending* := *pending* $\cup$ $\{(self, m)\}$;
**trigger** $\langle$ *beb*, *Broadcast* | [DATA, *self*, $m$] $\rangle$;

**upon event** $\langle$ *beb*, *Deliver* | $p$, [DATA, $s, m$] $\rangle$ **do**
*ack*$[m]$ := *ack*$[m]$ $\cup$ $\{p\}$;
**if** $(s, m) \notin$ *pending* **then**
*pending* := *pending* $\cup$ $\{(s, m)\}$;
**trigger** $\langle$ *beb*, *Broadcast* | [DATA, $s, m$] $\rangle$;

**function** *candeliver*$(m)$ **returns** Boolean **is**
**return** $\#(ack[m]) > N/2$;

**upon exists** $(s, m) \in$ *pending* such that *candeliver*$(m) \wedge m \notin$ *delivered* **do**
*delivered* := *delivered* $\cup$ $\{m\}$;
**trigger** $\langle$ *urb*, *Deliver* | $s, m$ $\rangle$;

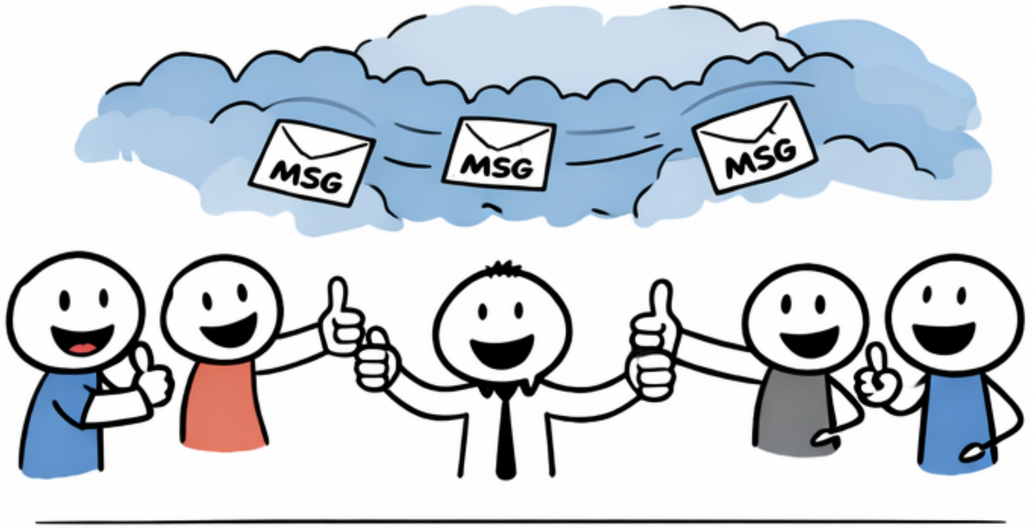
# Joke time!

# All in or nothing

# No message left behind

# Uniform or it didn't happen


Reliable Broadcast




Uniform Reliable Broadcast


Reliable broadcast... but not uniform


Uniform reliable broadcast!

# Skipping ahead

# Read on your own

- Read on your own
  - 3.5 Stubborn Broadcast
  - 3.6 Logged Best-Effort Broadcast
  - 3.7 Logged Uniform Reliable Broadcast
- Will be covered later
  - 3.8 Probabilistic Broadcast

# FIFO and Causal Broadcast

# Ch 3.9 FIFO and Causal Broadcast

## Reliable Broadcast

- Ensures *agreement* on delivered messages
- **Does NOT** specify *ordering* between messages from different processes
- **Key limitation**
  - Messages are treated as *independent*
  - Different processes may deliver messages *in different orders*

# With Reliable Broadcast, Ordering becomes an Application Concern

# Want to Provide Ordering Abstraction

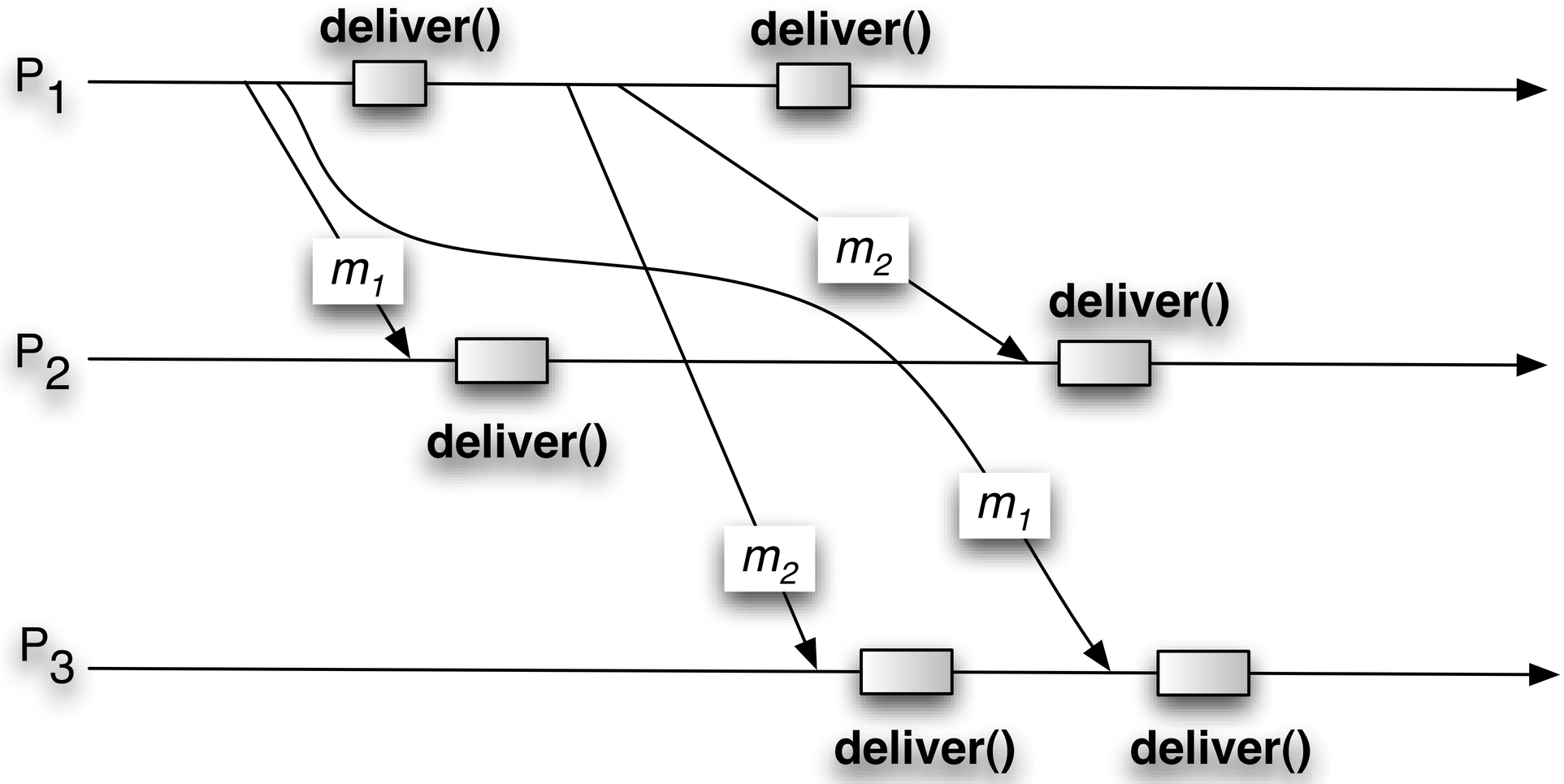
# Ch 3.9 FIFO and Causal Broadcast

## FIFO Broadcast: First Step Toward Order

- Problem
  - Two messages broadcast by the same process
  - May be delivered in different orders at different processes

# Ch 3.9 FIFO and Causal Broadcast

## FIFO Ordering Violation



# Ch 3.9 FIFO and Causal Broadcast

## FIFO Broadcast: First Step Toward Order

- FIFO Broadcast
  - Ensures messages from the **same sender** are **delivered in broadcast order**
  - *No guarantees across different senders*
- Observation
  - FIFO violations are **local**
  - Solvable using **sequence numbers**

# Ch 3.9 FIFO and Causal Broadcast

**Module 3.8:** Interface and properties of FIFO-order (reliable) broadcast

**Module:**

**Name:** FIFOReliableBroadcast, **instance** *frb*.

**Events:**

**Request:** $\langle$ *frb*, *Broadcast* | $m$ $\rangle$: Broadcasts a message $m$ to all processes.

**Indication:** $\langle$ *frb*, *Deliver* | $p$, $m$ $\rangle$: Delivers a message $m$ broadcast by process $p$.

**Properties:**

**FRB1–FRB4:** Same as properties RB1–RB4 in (regular) reliable broadcast (Module 3.2).

**FRB5:** *FIFO delivery:* If some process broadcasts message $m_1$ before it broadcasts message $m_2$, then no correct process delivers $m_2$ unless it has already delivered $m_1$.

# Ch 3.9 FIFO and Causal Broadcast

Whiteboard

**Algorithm 3.12:** Broadcast with Sequence Number

**Implements:**
FIFOReliableBroadcast, **instance** *frb*.

**Uses:**
ReliableBroadcast, **instance** *rb*.

**upon event** $\langle$ *frb*, *Init* $\rangle$ **do**
*lsn* := 0;
*pending* := $\emptyset$;
*next* := $[1]^N$;

**upon event** $\langle$ *frb*, *Broadcast* | $m$ $\rangle$ **do**
*lsn* := *lsn* + 1;
**trigger** $\langle$ *rb*, *Broadcast* | [DATA, *self*, $m$, *lsn*] $\rangle$;

**upon event** $\langle$ *rb*, *Deliver* | $p$, [DATA, $s, m, sn$] $\rangle$ **do**
*pending* := *pending* $\cup \{(s, m, sn)\}$;
**while exists** $(s, m', sn') \in$ *pending* such that $sn' =$ *next*$[s]$ **do**
*next*$[s]$ := *next*$[s] + 1$;
*pending* := *pending* $\setminus \{(s, m', sn')\}$;
**trigger** $\langle$ *frb*, *Deliver* | $s, m'$ $\rangle$;

FIFO is cheap, intuitive, and often "good enough"

— but it misses important dependencies

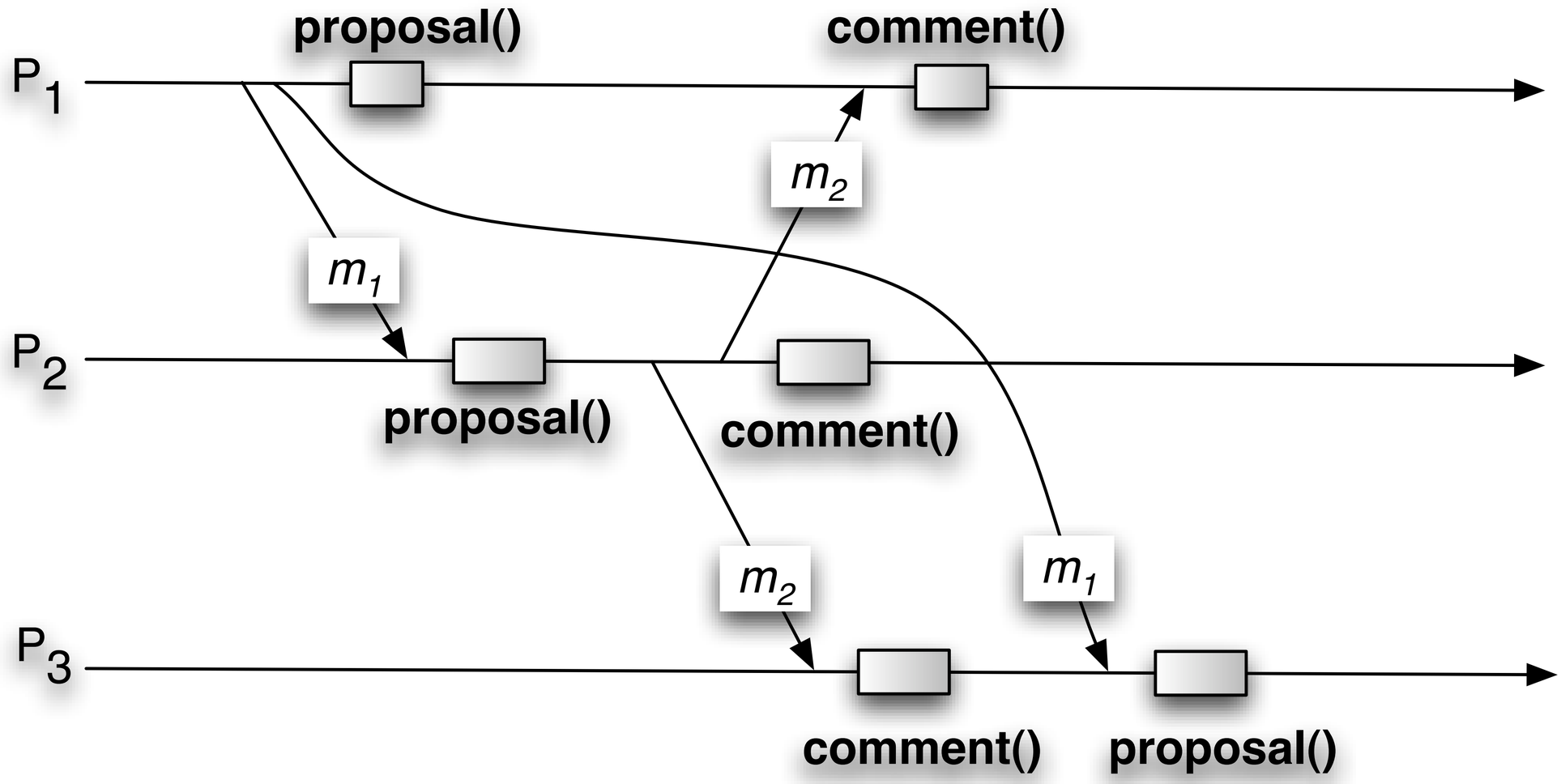
# Ch 3.9 FIFO and Causal Broadcast

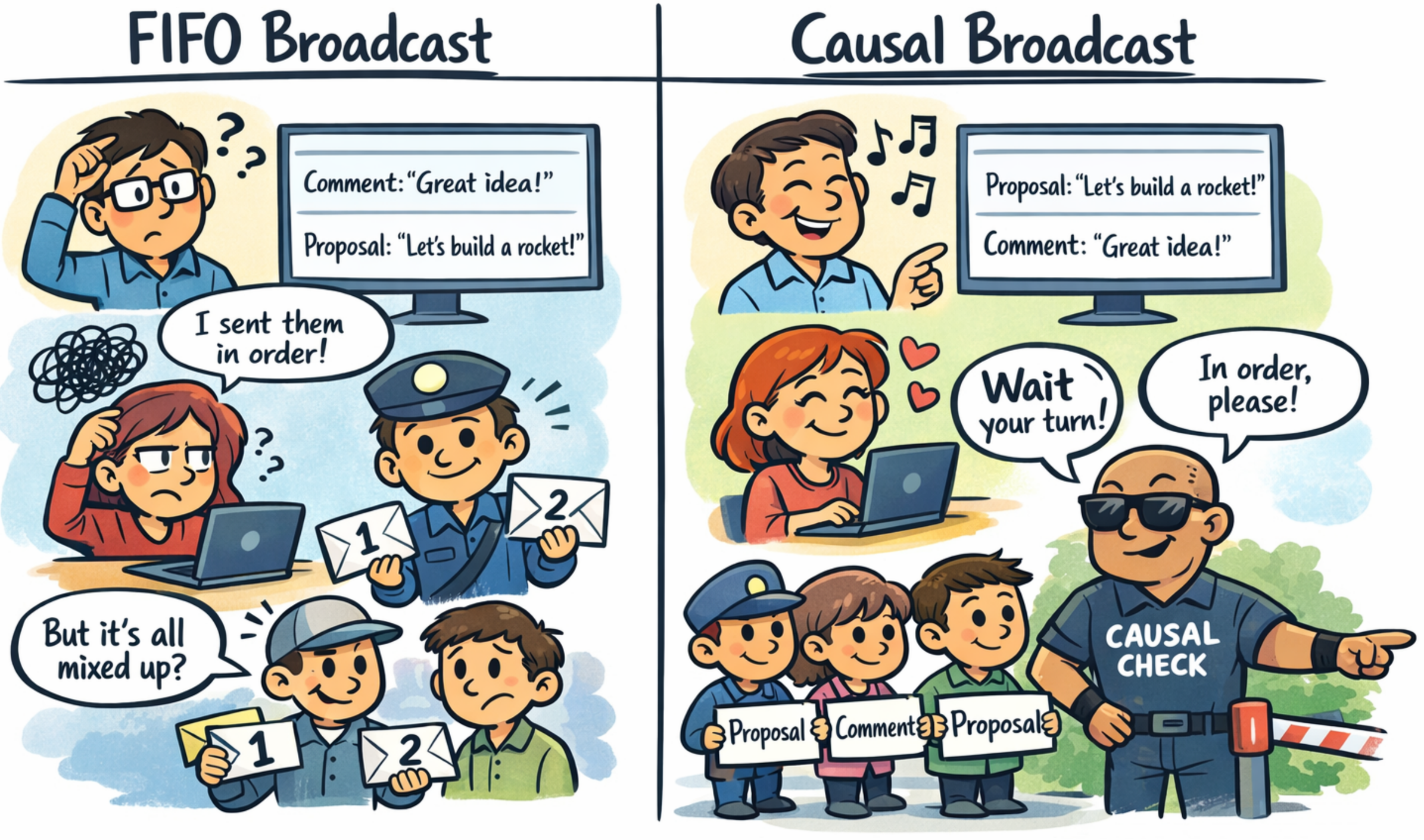
## Example: Distributed Message Board

- Participants broadcast:
  - Proposals
  - Comments on proposals
- Using RB
  - *May deliver comments before the proposal*

# Ch 3.9 FIFO and Causal Broadcast

## Example: Distributed Message Board




FIFO Broadcast
Comment: "Great idea!"
Proposal: "Let's build a rocket!"
I sent them in order!
1
2
But it's all mixed up?
1
2
Causal Broadcast
Proposal: "Let's build a rocket!"
Comment: "Great idea!"
Wait your turn!
In order, please!
CAUSAL CHECK
Proposal
Comment
Proposal

# Causal Order Broadcast

# Ch 3.9 FIFO and Causal Broadcast

## Causal Broadcast: Goal

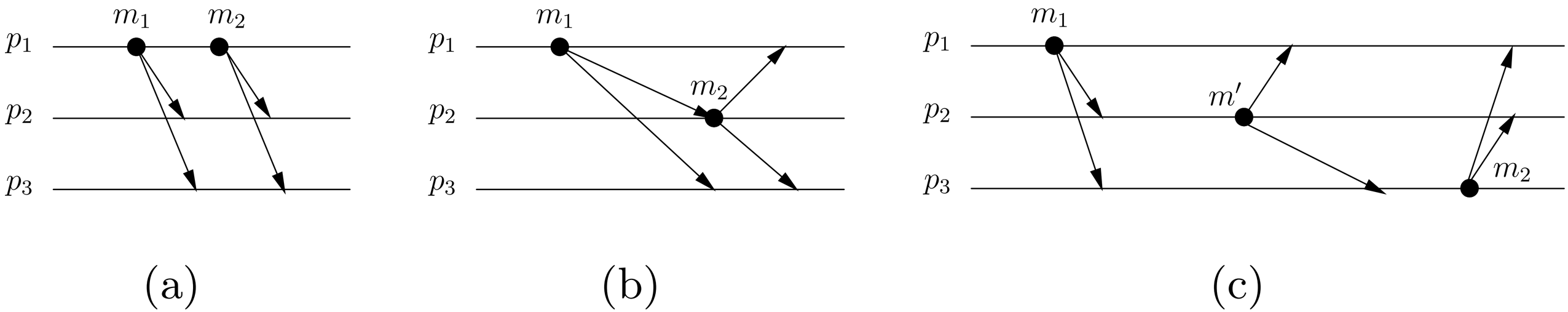
- Ensure messages are **delivered respecting cause–effect relations**
- Causal Order
  - If *m1 may have **caused** m2*, then:
  - Every process delivers m1 before m2
- Key idea
  - **Ordering follows information flow**, not sender identity

# Causal Order Abstraction ensures Cause-effect Delivery

# Ch 3.9 FIFO and Causal Broadcast

## Causal Ordering Relation: Definition

- A message m1 may (potentially) have caused another message m2 if one of the following relations apply:



# Ch 3.9 FIFO and Causal Broadcast

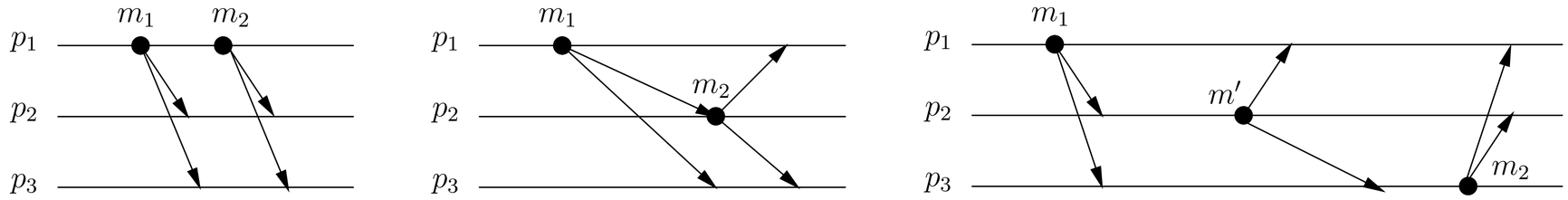
## Causal Ordering Relation: Definition

- A message m1 may (potentially) have caused another message m2 if one of the following relations apply:

  (a) broadcast(m1) → broadcast(m2) (Same process)

  (b) deliver(m1) → broadcast(m2) (Message induces message)

  (c) ∃m′: m1 → m′ ∧ m′ → m2 ⇒ m1 → m2 (Transitivity)



# Ch 3.9 FIFO and Causal Broadcast

**Module 3.9:** Interface and properties of causal-order (reliable) broadcast

**Module:**

**Name:** CausalOrderReliableBroadcast, **instance** *crb*.

**Events:**

**Request:** $\langle$ *crb*, *Broadcast* | $m$ $\rangle$: Broadcasts a message $m$ to all processes.

**Indication:** $\langle$ *crb*, *Deliver* | $p$, $m$ $\rangle$: Delivers a message $m$ broadcast by process $p$.

**Properties:**

**CRB1–CRB4:** Same as properties RB1–RB4 in (regular) reliable broadcast (Module 3.2).

**CRB5:** *Causal delivery:* For any message $m_1$ that potentially caused a message $m_2$, i.e., $m_1 \rightarrow m_2$, no process delivers $m_2$ unless it has already delivered $m_1$.

# Ch 3.9 FIFO and Causal Broadcast

## Causal Broadcast Property (CRB5)

- No holes in the causal past
- When a message m is delivered:
  - *All messages that causally precede m have already been delivered*
- ➡️ Applications **never observe effects before causes**

# Ch 3.9 FIFO and Causal Broadcast

- Fail-silent model
- No failure detector
- Past field carries entire *causal history*
- Immediate crbDeliver(m)
  - No waiting

**Algorithm 3.13:** No-Waiting Causal Broadcast

**Implements:**
CausalOrderReliableBroadcast, **instance** *crb*.

**Uses:**
ReliableBroadcast, **instance** *rb*.

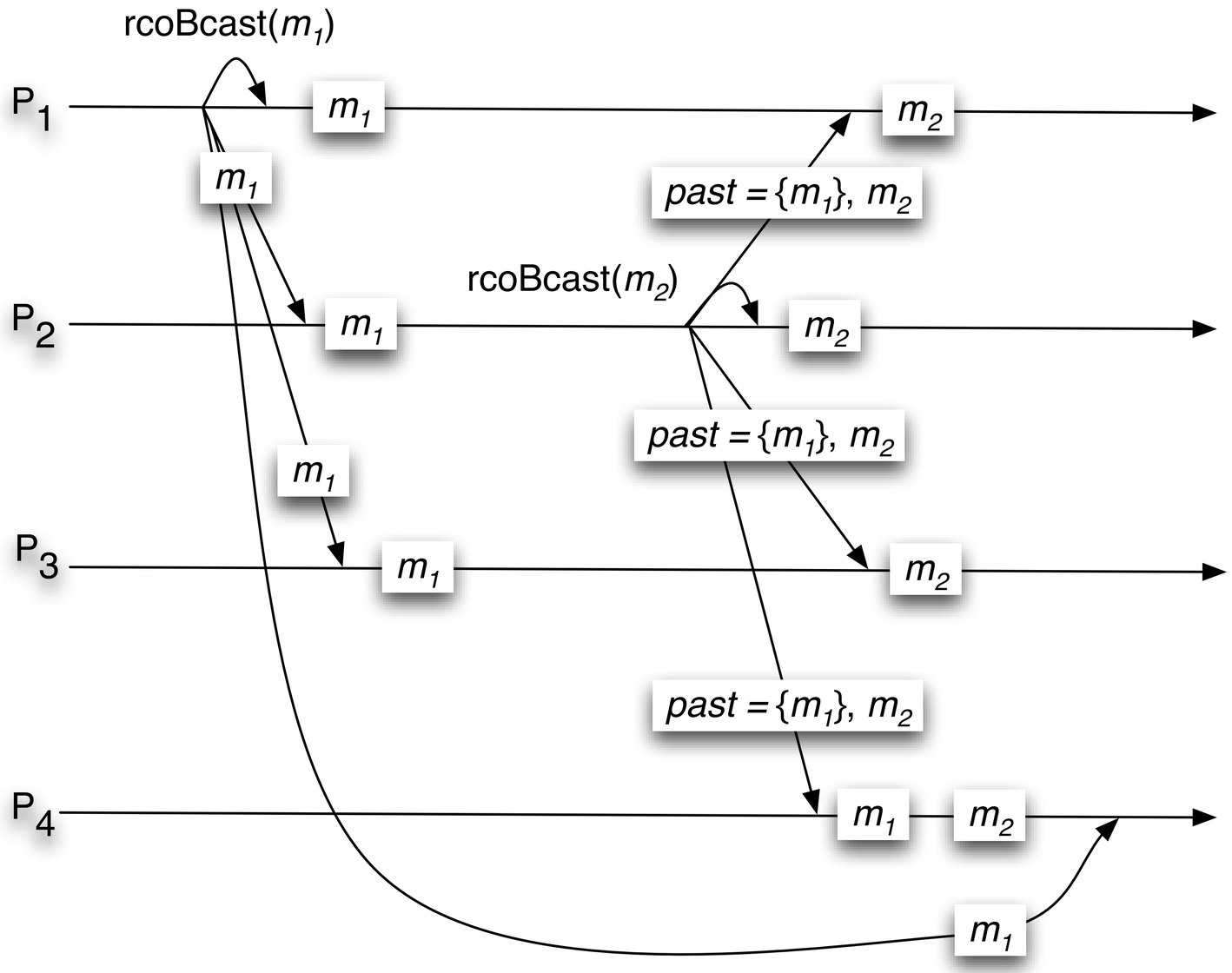
**upon event** $\langle$ *crb*, *Init* $\rangle$ **do**
  *delivered* := $\emptyset$;
  *past* := [];

**upon event** $\langle$ *crb*, *Broadcast* | $m$ $\rangle$ **do**
  **trigger** $\langle$ *rb*, *Broadcast* | [DATA, *past*, $m$] $\rangle$;
  *append*(*past*, (*self*, $m$));

**upon event** $\langle$ *rb*, *Deliver* | $p$, [DATA, *mpast*, $m$] $\rangle$ **do**
  **if** $m \notin$ *delivered* **then**
    **forall** $(s, n) \in$ *mpast* **do**
      **if** $n \notin$ *delivered* **then**
        **trigger** $\langle$ *crb*, *Deliver* | $s, n$ $\rangle$;
        *delivered* := *delivered* $\cup \{n\}$;
        **if** $(s, n) \notin$ *past* **then**
          *append*(*past*, $(s, n)$);
    **trigger** $\langle$ *crb*, *Deliver* | $p$, $m$ $\rangle$;
    *delivered* := *delivered* $\cup \{m\}$;
    **if** $(p, m) \notin$ *past* **then**
      *append*(*past*, $(p, m)$);

# Ch 3.9 FIFO and Causal Broadcast

Whiteboard




**upon event** $\langle$ *crb*, *Init* $\rangle$ **do**
    $delivered := \emptyset$;
    $past := []$;

**upon event** $\langle$ *crb*, *Broadcast* | $m$ $\rangle$ **do**
    **trigger** $\langle$ *rb*, *Broadcast* | [DATA, *past*, $m$] $\rangle$;
    *append*(*past*, (*self*, $m$));

**upon event** $\langle$ *rb*, *Deliver* | $p$, [DATA, *mpast*, $m$] $\rangle$ **do**
    **if** $m \notin delivered$ **then**
        **forall** $(s, n) \in mpast$ **do**
            **if** $n \notin delivered$ **then**
                **trigger** $\langle$ *crb*, *Deliver* | $s, n$ $\rangle$;
                $delivered := delivered \cup \{n\}$;
                **if** $(s, n) \notin past$ **then**
                    *append*(*past*, $(s, n)$);
        **trigger** $\langle$ *crb*, *Deliver* | $p, m$ $\rangle$;
        $delivered := delivered \cup \{m\}$;
        **if** $(p, m) \notin past$ **then**
            *append*(*past*, $(p, m)$);

# Ch 3.9 FIFO and Causal Broadcast

## Performance

- Communication steps as for RB
- Size of messages grow linearly with number of messages sent

**Algorithm 3.13:** No-Waiting Causal Broadcast

**Implements:**
CausalOrderReliableBroadcast, **instance** *crb*.

**Uses:**
ReliableBroadcast, **instance** *rb*.

**upon event** $\langle$ *crb*, *Init* $\rangle$ **do**
*delivered* := $\emptyset$;
*past* := [];

**upon event** $\langle$ *crb*, *Broadcast* | $m$ $\rangle$ **do**
**trigger** $\langle$ *rb*, *Broadcast* | [DATA, *past*, $m$] $\rangle$;
*append*(*past*, (*self*, $m$));

**upon event** $\langle$ *rb*, *Deliver* | $p$, [DATA, *mpast*, $m$] $\rangle$ **do**
**if** $m \notin$ *delivered* **then**
**forall** $(s, n) \in$ *mpast* **do**
**if** $n \notin$ *delivered* **then**
**trigger** $\langle$ *crb*, *Deliver* | $s, n$ $\rangle$;
*delivered* := *delivered* $\cup \{n\}$;
**if** $(s, n) \notin$ *past* **then**
*append*(*past*, $(s, n)$);
**trigger** $\langle$ *crb*, *Deliver* | $p$, $m$ $\rangle$;
*delivered* := *delivered* $\cup \{m\}$;
**if** $(p, m) \notin$ *past* **then**
*append*(*past*, $(p, m)$);

# Ch 3.9 FIFO and Causal Broadcast

- Fail-stop model
- Uses perfect failure detector
- ACK on delivery
- Purge messages after all ACKs
- Distributed garbage collection

**Algorithm 3.14:** Garbage-Collection of Causal Past (extends Algorithm 3.13)

**Implements:**
CausalOrderReliableBroadcast, **instance** *crb*.

**Uses:**
ReliableBroadcast, **instance** *rb*;
PerfectFailureDetector, **instance** $\mathcal{P}$.

// Except for its ⟨ *Init* ⟩ event handler, the pseudo code of Algorithm 3.13 is also
// part of this algorithm.

**upon event** ⟨ *crb*, *Init* ⟩ **do**
*delivered* := ∅;
*past* := [];
*correct* := $\Pi$;
**forall** $m$ **do** *ack*[$m$] := ∅;

**upon event** ⟨ $\mathcal{P}$, *Crash* | $p$ ⟩ **do**
*correct* := *correct* \ {$p$};

**upon exists** $m \in$ *delivered* such that *self* $\notin$ *ack*[$m$] **do**
*ack*[$m$] := *ack*[$m$] ∪ {*self*};
**trigger** ⟨ *rb*, *Broadcast* | [ACK, $m$] ⟩;

**upon event** ⟨ *rb*, *Deliver* | $p$, [ACK, $m$] ⟩ **do**
*ack*[$m$] := *ack*[$m$] ∪ {$p$};

**upon** *correct* ⊆ *ack*[$m$] **do**
**forall** $(s', m') \in$ *past* such that $m' = m$ **do**
remove(*past*, $(s', m)$);

# Ch 3.9 FIFO and Causal Broadcast

## Tradeoff

- Use failure detector
- Get bounded message size

**Algorithm 3.14:** Garbage-Collection of Causal Past (extends Algorithm 3.13)

**Implements:**
CausalOrderReliableBroadcast, **instance** *crb*.

**Uses:**
ReliableBroadcast, **instance** *rb*;
PerfectFailureDetector, **instance** $\mathcal{P}$.

// Except for its ⟨ *Init* ⟩ event handler, the pseudo code of Algorithm 3.13 is also
// part of this algorithm.

**upon event** ⟨ *crb*, *Init* ⟩ **do**
  $delivered := \emptyset$;
  $past := []$;
  $correct := \Pi$;
  **forall** $m$ **do** $ack[m] := \emptyset$;

**upon event** ⟨ $\mathcal{P}$, *Crash* | $p$ ⟩ **do**
  $correct := correct \setminus \{p\}$;

**upon exists** $m \in delivered$ such that $self \notin ack[m]$ **do**
  $ack[m] := ack[m] \cup \{self\}$;
  **trigger** ⟨ *rb*, *Broadcast* | [ACK, $m$] ⟩;

**upon event** ⟨ *rb*, *Deliver* | $p$, [ACK, $m$] ⟩ **do**
  $ack[m] := ack[m] \cup \{p\}$;

**upon** $correct \subseteq ack[m]$ **do**
  **forall** $(s', m') \in past$ such that $m' = m$ **do**
    $remove(past, (s', m))$;

# Causal Order Broadcast Vector Clock Optimization

# Ch 3.9 FIFO and Causal Broadcast

- Fail-silent model
- No failure detector
- Maintain vector of seq#s per process
- Attach vectors to each message
- Receiver compares vectors

**Algorithm 3.15:** Waiting Causal Broadcast

**Implements:**
CausalOrderReliableBroadcast, **instance** *crb*.

**Uses:**
ReliableBroadcast, **instance** *rb*.

**upon event** $\langle$ *crb*, *Init* $\rangle$ **do**
$V := [0]^N$;
$lsn := 0$;
$pending := \emptyset$;

**upon event** $\langle$ *crb*, *Broadcast* | $m$ $\rangle$ **do**
$W := V$;
$W[rank(self)] := lsn$;
$lsn := lsn + 1$;
**trigger** $\langle$ *rb*, *Broadcast* | [DATA, $W, m$] $\rangle$;

**upon event** $\langle$ *rb*, *Deliver* | $p$, [DATA, $W, m$] $\rangle$ **do**
$pending := pending \cup \{(p, W, m)\}$;
**while exists** $(p', W', m') \in pending$ such that $W' \leq V$ **do**
$pending := pending \setminus \{(p', W', m')\}$;
$V[rank(p')] := V[rank(p')] + 1$;
**trigger** $\langle$ *crb*, *Deliver* | $p', m'$ $\rangle$;

# Ch 3.9 FIFO and Causal Broadcast

## Tradeoff

- Wait for missing messages
- No retransmission requests

**Algorithm 3.15:** Waiting Causal Broadcast

**Implements:**
CausalOrderReliableBroadcast, **instance** *crb*.

**Uses:**
ReliableBroadcast, **instance** *rb*.

**upon event** $\langle$ *crb*, *Init* $\rangle$ **do**
$V := [0]^N$;
$lsn := 0$;
$pending := \emptyset$;

**upon event** $\langle$ *crb*, *Broadcast* | $m$ $\rangle$ **do**
$W := V$;
$W[rank(self)] := lsn$;
$lsn := lsn + 1$;
**trigger** $\langle$ *rb*, *Broadcast* | [DATA, $W, m$] $\rangle$;

**upon event** $\langle$ *rb*, *Deliver* | $p$, [DATA, $W, m$] $\rangle$ **do**
$pending := pending \cup \{(p, W, m)\}$;
**while exists** $(p', W', m') \in pending$ such that $W' \leq V$ **do**
$pending := pending \setminus \{(p', W', m')\}$;
$V[rank(p')] := V[rank(p')] + 1$;
**trigger** $\langle$ *crb*, *Deliver* | $p', m'$ $\rangle$;

# Ch 3.9 FIFO and Causal Broadcast

## Vector Clocks

- Key idea
  - Represent ***causal past compactly using a vector clock***
- **Vector Clock** (VC)
  - Array of integers
  - One entry per process
  - VC[ *i* ] = *number of messages delivered* from **process *i***
- rank( $p_i$ ) = *i*

# Ch 3.9 FIFO and Causal Broadcast

## Vector Clocks Broadcast

- Each process p maintains VC
  - **Increment own entry** when broadcasting
  - Attach VC to every broadcast message
- At receiver q
  - **Compare received VC with local VC**
  - **Detect missing messages**

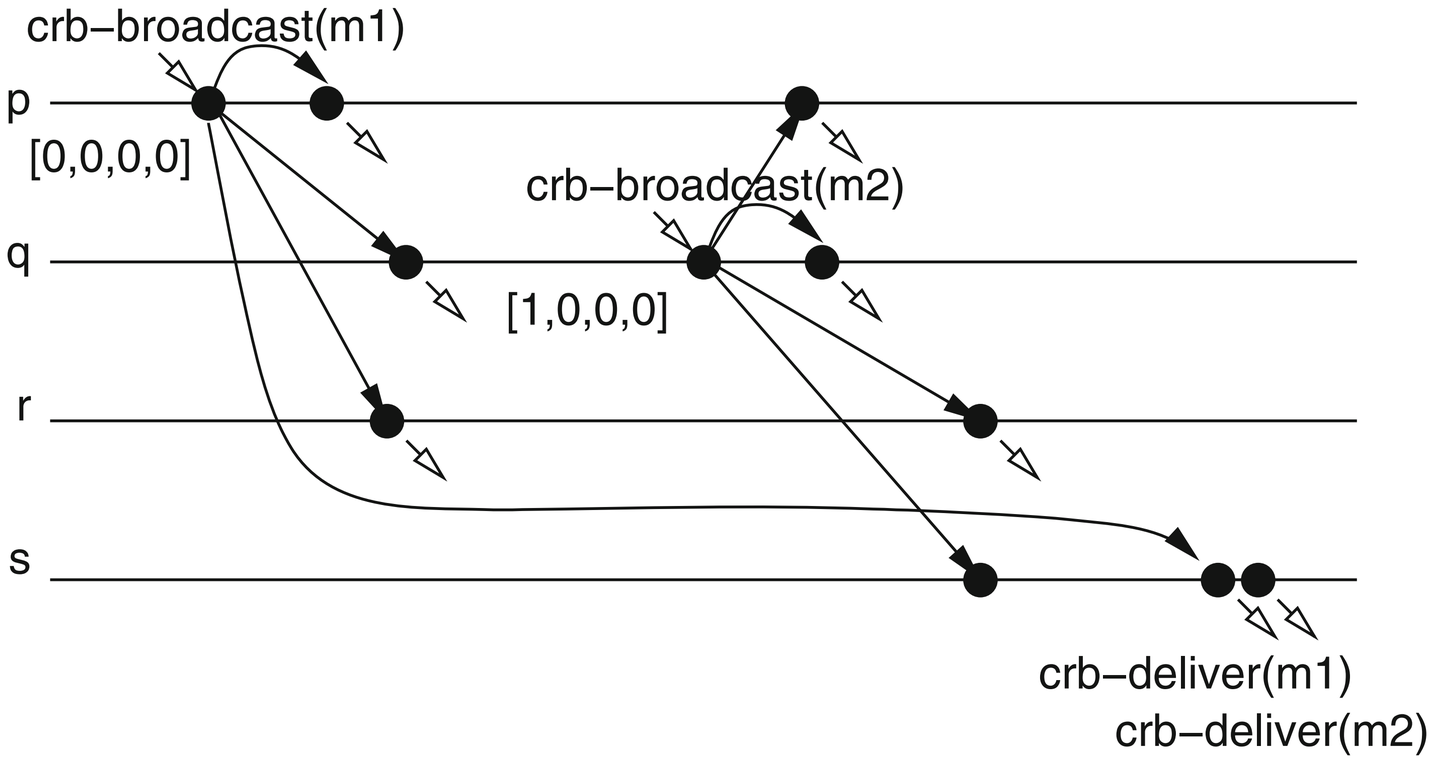
# Ch 3.9 FIFO and Causal Broadcast

## Vector Clocks Broadcast: Delivery Rule

- On rbDeliver(m)
  - If **all causally prior messages** have been delivered: crbDeliver(m)
- Else:
  - **Delay delivery** until missing messages arrive
  - (RB will eventually deliver missing messages)

# Ch 3.9 FIFO and Causal Broadcast

## Whiteboard




**Algorithm 3.15:** Waiting Causal Broadcast

**Implements:**

CausalOrderReliableBroadcast, **instance** *crb*.

**Uses:**

ReliableBroadcast, **instance** *rb*.

**upon event** $\langle$ *crb*, *Init* $\rangle$ **do**

$V := [0]^N$;
$lsn := 0$;
$pending := \emptyset$;

**upon event** $\langle$ *crb*, *Broadcast* | $m$ $\rangle$ **do**

$W := V$;
$W[rank(self)] := lsn$;
$lsn := lsn + 1$;
**trigger** $\langle$ *rb*, *Broadcast* | $[\text{DATA}, W, m]$ $\rangle$;

**upon event** $\langle$ *rb*, *Deliver* | $p$, $[\text{DATA}, W, m]$ $\rangle$ **do**

$pending := pending \cup \{(p, W, m)\}$;
**while exists** $(p', W', m') \in pending$ such that $W' \leq V$ **do**
$pending := pending \setminus \{(p', W', m')\}$;
$V[rank(p')] := V[rank(p')] + 1$;
**trigger** $\langle$ *crb*, *Deliver* | $p'$, $m'$ $\rangle$;

# Ch 3.9 FIFO and Causal Broadcast

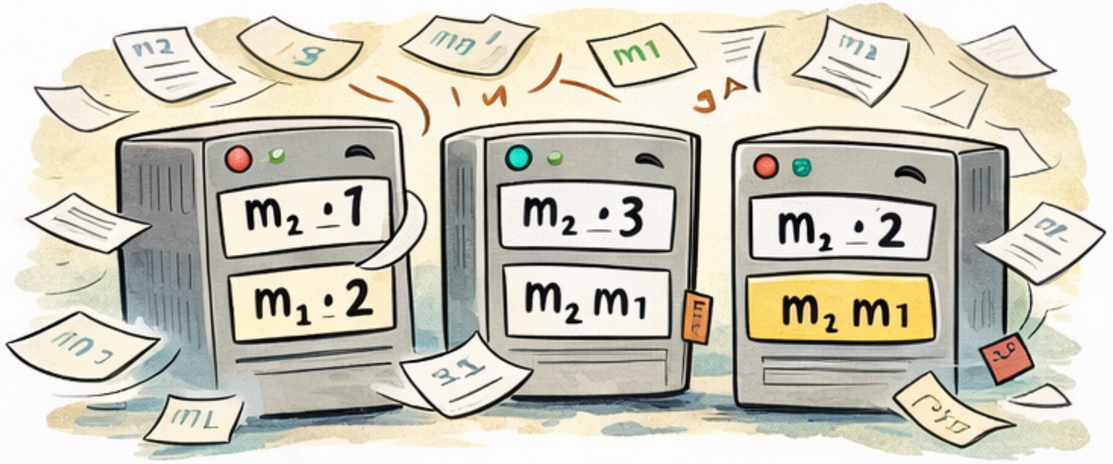
## Takeaway

- RB: Agreement, **no order**
- FIFO: **Sender order** only
- Causal Broadcast: **Causal order**
  - Matches application semantics
  - Eliminates ordering logic from apps

# Causal broadcast lets applications stay simple

## Reliable Broadcast:

Everyone gets the messages...

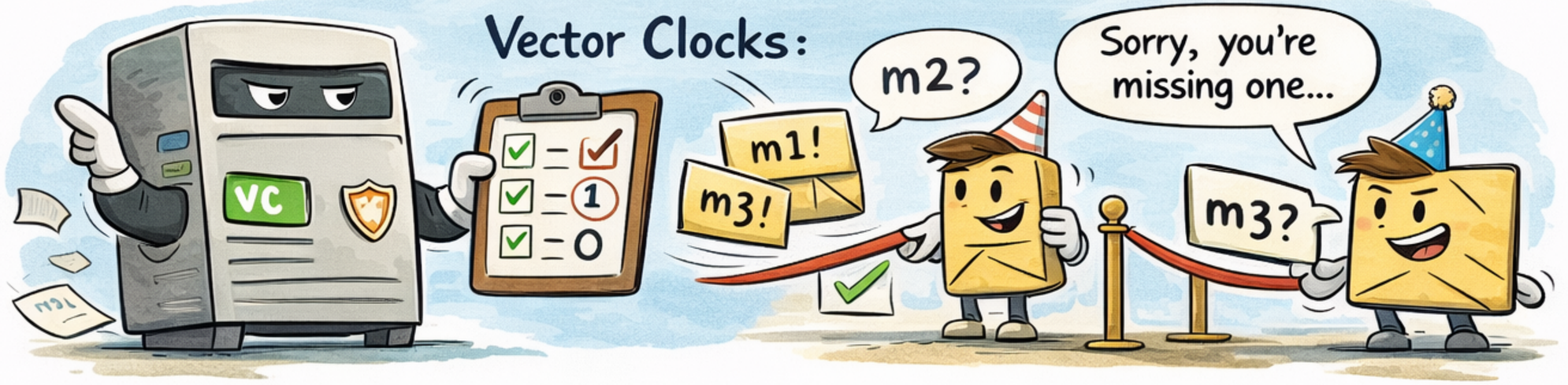



Everyone gets the messages...
...eventually... in some order.

## FIFO promotes good manners...
but not good sense.


FIFO promotes good manners...
... but not good sense.

## Vector Clocks:




**Causal watchdogs with zero tolerance !**